

Queue & Deque — Complete Guide

1. Queue Basics

1. Overview

A **Queue** is a linear data structure that follows the **FIFO** (First In, First Out) principle. The element that is inserted first is the one that gets removed first. Think of a queue of people at a ticket counter — the person who stands first gets served first.

Queues are used everywhere: CPU scheduling, BFS traversal, print spooling, request handling, and as a building block for many algorithms.

2. Intuition

Imagine a line at a movie theatre. People join at the back and leave from the front. You cannot cut in the middle — you must wait your turn. This is exactly how a queue works.

- **Enqueue** → a person joins at the back.
- **Dequeue** → the person at the front leaves.
- **Front** → the person who is next to leave.

The key insight: **order of insertion = order of processing**. This makes queues ideal for any "first-come, first-serve" scenario, or for processing things in the order they are discovered (like BFS).

3. When to Use It

- When you need to process elements in the order they arrive.
- When a problem asks for **level-order** or **BFS** traversal.
- When you need to **buffer** data between a producer and consumer.
- When you need to simulate a **real-world queue** (ticketing, requests, calls).
- When you see "**in order of arrival**" or "**first come first serve**" in the problem.
- When you need to implement a **cache** (like FIFO cache).

4. When Not to Use It

- When you need random access to elements (use a vector/array).
- When you need LIFO (use a stack).
- When you need priority-based processing (use a priority queue).
- When you need fast lookup/search (use a set or hashmap).
- When the queue size is fixed and small and you need simplicity — a simple array with front/rear pointers may suffice.

5. Core Concepts

Concept	Explanation	Why It Matters
FIFO	First In, First Out ordering	The fundamental invariant of a queue
Front	Pointer/index to the element that will be removed next	O(1) access to the next element
Rear/Back	Pointer/index where the next element will be inserted	O(1) insertion at the end
Enqueue (push)	Adding an element to the back of the queue	Basic insertion operation
Dequeue (pop)	Removing an element from the front of the queue	Basic removal operation
Empty	Queue has no elements	Must check before dequeue/front
Full	Queue has reached maximum capacity (in array-based queues)	Overflow condition

6. Step-by-Step Algorithm

Basic Queue Operations:

1. **Initialize** an empty queue with a fixed capacity or dynamic structure.
2. **Enqueue (x):**
 - If queue is full, report overflow.
 - Increment rear pointer (with wrap-around if circular).
 - Place **x** at **rear**.
3. **Dequeue ():**
 - If queue is empty, report underflow.
 - Retrieve element at **front**.
 - Increment front pointer (with wrap-around).
4. **Front ():**
 - If queue is empty, return error.
 - Return element at **front**.
5. **IsEmpty ():**
 - Return **true** if `front == rear` (or `size == 0`).

7. Dry Run

Operations: Enqueue(5), Enqueue(10), Enqueue(15), Dequeue(), Dequeue(), Enqueue(20), Front()

Step	Operation	Front	Rear	Queue (front → rear)	Size
1	Init	0	0	[]	0
2	Enqueue(5)	0	1	[5]	1

Step	Operation	Front	Rear	Queue (front → rear)	Size
3	Enqueue(10)	0	2	[5, 10]	2
4	Enqueue(15)	0	3	[5, 10, 15]	3
5	Dequeue() → 5	1	3	[10, 15]	2
6	Dequeue() → 10	2	3	[15]	1
7	Enqueue(20)	2	4	[15, 20]	2
8	Front() → 15	2	4	[15, 20]	2

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class Queue {
private:
    int *arr;
    int front, rear, capacity, size;

public:
    // Constructor
    Queue(int cap = 100000) {
        capacity = cap;
        arr = new int[capacity];
        front = 0;
        rear = 0;
        size = 0;
    }

    ~Queue() {
        delete[] arr;
    }

    // Enqueue – add element at rear
    void enqueue(int x) {
        if (size == capacity) {
            cout << "Queue Overflow\n";
            return;
        }
        arr[rear] = x;
        rear = (rear + 1) % capacity;
        size++;
    }

    // Dequeue – remove element from front
    int dequeue() {
        if (isEmpty()) {
            cout << "Queue Underflow\n";
        }
    }
};
```

```
        return -1;
    }
    int val = arr[front];
    front = (front + 1) % capacity;
    size--;
    return val;
}

// Get front element
int getFront() {
    if (isEmpty()) {
        cout << "Queue is Empty\n";
        return -1;
    }
    return arr[front];
}

// Get rear element
int getRear() {
    if (isEmpty()) {
        cout << "Queue is Empty\n";
        return -1;
    }
    return arr[(rear - 1 + capacity) % capacity];
}

bool isEmpty() {
    return size == 0;
}

int getSize() {
    return size;
}
};

// --- Example Usage ---
int main() {
    Queue q(5);

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    cout << "Front: " << q.getFront() << "\n"; // 10
    cout << "Rear: " << q.getRear() << "\n";    // 30

    cout << "Dequeued: " << q.dequeue() << "\n"; // 10
    cout << "Dequeued: " << q.dequeue() << "\n"; // 20

    q.enqueue(40);
    cout << "Front: " << q.getFront() << "\n"; // 30
}
```

```
    return 0;
}
```

9. Python Implementation

```
class Queue:
    def __init__(self, capacity=100000):
        self.arr = [0] * capacity
        self.capacity = capacity
        self.front = 0
        self.rear = 0
        self.size = 0

    def enqueue(self, x):
        if self.size == self.capacity:
            print("Queue Overflow")
            return
        self.arr[self.rear] = x
        self.rear = (self.rear + 1) % self.capacity
        self.size += 1

    def dequeue(self):
        if self.is_empty():
            print("Queue Underflow")
            return -1
        val = self.arr[self.front]
        self.front = (self.front + 1) % self.capacity
        self.size -= 1
        return val

    def get_front(self):
        if self.is_empty():
            return -1
        return self.arr[self.front]

    def get_rear(self):
        if self.is_empty():
            return -1
        return self.arr[(self.rear - 1 + self.capacity) % self.capacity]

    def is_empty(self):
        return self.size == 0

    def get_size(self):
        return self.size

# Example usage
if __name__ == "__main__":
    q = Queue(5)
    q.enqueue(10)
```

```
q.enqueue(20)
q.enqueue(30)
print("Front:", q.get_front()) # 10
print("Rear:", q.get_rear())   # 30
print("Dequeued:", q.dequeue()) # 10
print("Dequeued:", q.dequeue()) # 20
q.enqueue(40)
print("Front:", q.get_front())  # 30
```

10. Code Explanation

- **Array-based storage:** We use a circular array to avoid shifting elements. The **front** and **rear** pointers wrap around using modulo.
- **Enqueue:** Places element at **rear**, then increments **rear** circularly. Increases **size**.
- **Dequeue:** Reads element at **front**, increments **front** circularly. Decreases **size**.
- **isEmpty:** Checks if **size == 0**. This is cleaner than checking **front == rear** because after wrap-around, **front == rear** could also mean a full queue.
- **Overflow/Underflow:** We check **size == capacity** before enqueue and **size == 0** before dequeue.
- **getRear:** Since **rear** always points to the next empty slot, the actual rear element is at **(rear - 1 + capacity) % capacity**.

11. Complexity Analysis

Operation	Time Complexity	Space Complexity
Enqueue	O(1)	O(1)
Dequeue	O(1)	O(1)
Front	O(1)	O(1)
Rear	O(1)	O(1)
isEmpty	O(1)	O(1)
Size	O(1)	O(1)
Overall (n elements)	O(1) per op	O(n)

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
BFS Traversal	Graph/tree level-order, shortest path in unweighted graph	Use queue to process nodes level by level	Level Order Traversal, Word Ladder
Producer-Consumer	Asynchronous processing, buffering	Queue acts as buffer between producer and consumer	Print Spooler, Task Queue

Pattern	How to Identify	Approach	Example Problems
Sliding Window	"Window of size k", "consecutive elements"	Queue holds window elements, front is oldest	Sliding Window Maximum, First Negative in Window
FIFO Cache	Cache with limited capacity, "least recent"	Queue tracks insertion order; evict from front	Page Replacement (FIFO)
Simulation	"Process in order of arrival", "event-driven"	Queue holds events/processes to be handled sequentially	Bank Queue Simulation, CPU Scheduling

13. Common Mistakes

- **Not checking if queue is empty** before dequeue/front → causes errors.
- **Confusing front and rear** during enqueue/dequeue.
- **Using simple array without circular wrap** → wasted space or $O(n)$ shift cost.
- **Incorrect modulo arithmetic** for circular queue → off-by-one errors.
- **Forgetting that rear points to next free slot**, not the last element.
- **Using front == rear to check emptiness** in a circular queue with size tracking — this can also mean the queue is full.
- **Not handling dynamic resizing** when using static array.

14. Edge Cases

- Empty queue: dequeue, front, rear should all return error.
- Single element: enqueue then dequeue should leave queue empty.
- Full queue: enqueue should report overflow.
- Wrap-around: after many operations, front and rear wrap correctly.
- Large capacity: ensure no integer overflow in size tracking.
- All operations interleaved: enqueue-dequeue-enqueue-dequeue pattern.

15. Variations

Variation	What Changes	When Used	Importance
Circular Queue	Rear wraps around to front when space is available at the beginning	Efficient fixed-size queue, OS buffers	High — placement favourite
Deque (Double-ended Queue)	Insert/delete from both ends	Sliding window max, palindrome checking	High — CP and placements
Priority Queue	Elements are dequeued based on priority, not insertion order	Dijkstra, Huffman coding, task scheduling	High — very common
Blocking Queue	Thread-safe queue with blocking enqueue/dequeue	Multithreaded producer-consumer	Moderate — system design

Variation	What Changes	When Used	Importance
Linked List Queue	Dynamic size, no wrap-around needed	When max size is unknown	Low — basic CS knowledge

16. Related Algorithms / Data Structures

Structure	Relationship	How to Choose
Stack	LIFO vs FIFO	Use queue when order matters; stack for reverse order
Deque	Queue with both ends accessible	Use deque when you need to add/remove from both ends
Priority Queue	Queue with priority ordering	Use when elements have different priorities
Array/Vector	Random access vs sequential	Use array for index-based access; queue for FIFO
Linked List	Dynamic linear structure	Linked list can implement queue, but array is more cache-friendly

17. Practice Problems

Easy

- **Implement Queue using Stacks** — LeetCode (232) — Use two stacks to simulate queue. Easy.
- **Number of Recent Calls** — LeetCode (933) — Sliding window counter using queue. Easy.

Medium

- **Time Needed to Buy Tickets** — LeetCode (2073) — Queue simulation. Medium.
- **Process Tasks Using Servers** — LeetCode (1882) — Two queues + priority queue. Medium.

Hard

- **Number of Visible People in a Queue** — LeetCode (1944) — Monotonic stack meets queue. Hard.
- **Minimum Time to Make Array Sum Zero** — Queue + greedy simulation. Hard.

18. Interview Explanation

"A queue is a linear data structure that follows the First-In-First-Out principle. Elements are added at the rear and removed from the front. The main operations are enqueue ($O(1)$), dequeue ($O(1)$), and front ($O(1)$). Queues are used anytime you need to process things in order of arrival — BFS in graphs, level-order tree traversal, CPU scheduling, and sliding window problems. An array-based circular queue is the most common implementation, where front and rear pointers wrap around to avoid wasted space."

19. Revision Notes

- **Key idea:** FIFO — first in, first out.

- **Operations:** enqueue (back), dequeue (front), front, rear, isEmpty — all $O(1)$.
- **Implementation:** circular array with front/rear pointers + size counter.
- **Common gotcha:** use `size` to check empty/full, not `front == rear`.
- **Use cases:** BFS, level-order, sliding window, simulation, buffering.
- **Complexity:** $O(1)$ per operation, $O(n)$ space.

20. Final Cheat Sheet

QUEUE

When to use: FIFO order, BFS, level-order, simulation

Main ops: enqueue(x), dequeue(), front(), isEmpty()

Complexity: $O(1)$ each op, $O(n)$ space

Key code: `arr[rear] = x; rear = (rear+1)%cap;`
`val = arr[front]; front = (front+1)%cap;`

Edge cases: empty queue, full queue, wrap-around

Trap: Don't use `front==rear` to check empty

2. Circular Queue

1. Overview

A **Circular Queue** is a fixed-size queue where the last position is connected back to the first position, forming a circle. This avoids the problem of wasted space that occurs in a normal linear queue when elements are dequeued from the front.

In a linear queue, after many dequeue operations, the front pointer moves forward, and the space before it becomes unusable. Circular queues solve this by reusing that space.

2. Intuition

Imagine a round table with chairs. People sit around the table. When someone leaves from one chair, that chair becomes available again. A new person can sit there. The seats never go to waste — they just cycle around.

In a circular queue:

- When `rear` reaches the end, it wraps to index 0 if space is available.
- When `front` reaches the end, it also wraps.
- The queue is full when `(rear + 1) % capacity == front`.
- The queue is empty when `front == rear`.

3. When to Use It

- When you have a **fixed-size buffer** and need to reuse space efficiently.

- **OS buffers** (keyboard buffer, I/O buffer, ring buffer).
- **Memory-constrained devices** where dynamic allocation is expensive.
- When you need **$O(1)$ enqueue/dequeue** with bounded space.
- **Sliding window** problems where the window is fixed.
- **Streaming data** processing (audio/video buffers).

4. When Not to Use It

- When you need a **dynamic (unbounded)** queue — use a linked-list queue.
- When the queue is **rarely full** and memory is not an issue — a simple queue is simpler.
- When you need **priority-based** ordering — use a priority queue.
- When you need **random access** to elements — use an array.
- When the **capacity is unknown** at compile time — dynamic allocation is simpler.

5. Core Concepts

Concept	Explanation	Why It Matters
Circular Wrap	$(\text{index} + 1) \% \text{capacity}$ to move to next position	Reuses space without shifting
Full Condition	$(\text{rear} + 1) \% \text{capacity} == \text{front}$	One slot is kept empty to distinguish full from empty
Empty Condition	$\text{front} == \text{rear}$	Both pointers are at same position
Size Tracking	Explicit <code>size</code> variable	Alternative to the one-empty-slot strategy
Ring Buffer	Same as circular queue, common in OS/systems	Used for audio/video buffers, I/O

6. Step-by-Step Algorithm

Using the one-empty-slot strategy (no size variable):

1. **Initialize** `front = 0`, `rear = 0`, array of size `capacity`.
2. **Enqueue (x):**
 - If $(\text{rear} + 1) \% \text{capacity} == \text{front}$ → queue is full, report overflow.
 - Place `x` at `rear`.
 - Set $\text{rear} = (\text{rear} + 1) \% \text{capacity}$.
3. **Dequeue ():**
 - If $\text{front} == \text{rear}$ → queue is empty, report underflow.
 - Read value at `front`.
 - Set $\text{front} = (\text{front} + 1) \% \text{capacity}$.
 - Return the value.
4. **Front ():**
 - If empty, return error.
 - Return `arr[front]`.
5. **Size ():**

- Return $(\text{rear} - \text{front} + \text{capacity}) \% \text{capacity}$.

7. Dry Run

Queue capacity = 5 (one slot kept empty, so max elements = 4).

Operations: Enqueue(1), Enqueue(2), Enqueue(3), Dequeue(), Enqueue(4), Enqueue(5), Dequeue(), Dequeue()

Step	Operation	Front	Rear	Queue (front → rear-1)	Full?
1	Init	0	0	[]	No
2	Enqueue(1)	0	1	[1]	No
3	Enqueue(2)	0	2	[1, 2]	No
4	Enqueue(3)	0	3	[1, 2, 3]	No
5	Dequeue() → 1	1	3	[2, 3]	No
6	Enqueue(4)	1	4	[2, 3, 4]	No
7	Enqueue(5)	1	0	[2, 3, 4, 5]	No (rear wraps to 0)
8	Dequeue() → 2	2	0	[3, 4, 5]	No
9	Dequeue() → 3	3	0	[4, 5]	No

Note: After step 7, $\text{arr}[0] = 5$ and $\text{rear} = 0$. The queue is nearly full: $(\text{rear} + 1) \% 5 = 1 == \text{front} \rightarrow$ would be full if we try to enqueue again.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class CircularQueue {
private:
    int *arr;
    int front, rear, capacity;

public:
    // Constructor – one slot is kept empty to distinguish full vs empty
    CircularQueue(int cap) {
        capacity = cap + 1; // one extra slot
        arr = new int[capacity];
        front = 0;
        rear = 0;
    }

    ~CircularQueue() {
        delete[] arr;
    }
}
```

```
bool isFull() {
    return (rear + 1) % capacity == front;
}

bool isEmpty() {
    return front == rear;
}

void enqueue(int x) {
    if (isFull()) {
        cout << "Queue is Full\n";
        return;
    }
    arr[rear] = x;
    rear = (rear + 1) % capacity;
}

int dequeue() {
    if (isEmpty()) {
        cout << "Queue is Empty\n";
        return -1;
    }
    int val = arr[front];
    front = (front + 1) % capacity;
    return val;
}

int getFront() {
    if (isEmpty()) {
        cout << "Queue is Empty\n";
        return -1;
    }
    return arr[front];
}

int getRear() {
    if (isEmpty()) {
        cout << "Queue is Empty\n";
        return -1;
    }
    return arr[(rear - 1 + capacity) % capacity];
}

int getSize() {
    return (rear - front + capacity) % capacity;
}
};

// --- Example Usage ---
int main() {
    CircularQueue cq(4); // can hold max 4 elements

    cq.enqueue(10);
    cq.enqueue(20);
```

```

cq.enqueue(30);
cq.enqueue(40);

cout << "Full: " << cq.isFull() << "\n"; // 1

cout << "Dequeued: " << cq.dequeue() << "\n"; // 10
cout << "Dequeued: " << cq.dequeue() << "\n"; // 20

cq.enqueue(50);
cout << "Front: " << cq.getFront() << "\n"; // 30
cout << "Rear: " << cq.getRear() << "\n"; // 50
cout << "Size: " << cq.getSize() << "\n"; // 3

return 0;
}

```

9. Python Implementation

```

class CircularQueue:
    def __init__(self, cap):
        self.capacity = cap + 1 # one extra slot
        self.arr = [0] * self.capacity
        self.front = 0
        self.rear = 0

    def is_full(self):
        return (self.rear + 1) % self.capacity == self.front

    def is_empty(self):
        return self.front == self.rear

    def enqueue(self, x):
        if self.is_full():
            print("Queue is Full")
            return
        self.arr[self.rear] = x
        self.rear = (self.rear + 1) % self.capacity

    def dequeue(self):
        if self.is_empty():
            print("Queue is Empty")
            return -1
        val = self.arr[self.front]
        self.front = (self.front + 1) % self.capacity
        return val

    def get_front(self):
        if self.is_empty():
            return -1
        return self.arr[self.front]

```

```

def get_rear(self):
    if self.is_empty():
        return -1
    return self.arr[(self.rear - 1 + self.capacity) % self.capacity]

def get_size(self):
    return (self.rear - self.front + self.capacity) % self.capacity

# Example usage
if __name__ == "__main__":
    cq = CircularQueue(4)
    cq.enqueue(10)
    cq.enqueue(20)
    cq.enqueue(30)
    cq.enqueue(40)
    print("Full:", cq.is_full()) # True
    print("Dequeued:", cq.dequeue()) # 10
    print("Dequeued:", cq.dequeue()) # 20
    cq.enqueue(50)
    print("Front:", cq.get_front()) # 30
    print("Rear:", cq.get_rear()) # 50
    print("Size:", cq.get_size()) # 3

```

10. Code Explanation

- **One-empty-slot strategy:** The array is allocated with `capacity + 1` slots. One slot is always kept empty. This allows us to distinguish between "full" and "empty" using just `front` and `rear` pointers.
- **isFull:** `(rear + 1) % capacity == front`. If advancing rear would land on front, the queue is full.
- **isEmpty:** `front == rear`. Both pointers at the same position.
- **getSize:** `(rear - front + capacity) % capacity`. This formula correctly handles wrap-around.
- **Enqueue:** Writes at `rear`, then advances `rear` circularly.
- **Dequeue:** Reads from `front`, then advances `front` circularly.
- **Alternative approach:** Use an explicit `size` variable instead of the one-empty-slot strategy. This uses all slots but adds one more variable.

11. Complexity Analysis

Operation	Time Complexity	Space Complexity
Enqueue	O(1)	O(1)
Dequeue	O(1)	O(1)
Front	O(1)	O(1)
Rear	O(1)	O(1)
isFull / isEmpty	O(1)	O(1)
Size	O(1)	O(1)

Operation	Time Complexity	Space Complexity
Overall	$O(1)$ per op	$O(\text{capacity})$

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Ring Buffer	Fixed-size buffer, streaming data, circular log	Circular queue with overwrite option	Audio buffer, log buffer
Sliding Window	Fixed window over a stream	Circular queue holds window elements	Moving average, rate limiter
Round-Robin Scheduling	CPU/process scheduling	Circular queue cycles through processes	OS scheduling

13. Common Mistakes

- **Off-by-one in capacity:** Forgetting the one-empty-slot requirement, so the usable capacity is `capacity - 1`.
- **Using `(rear + 1) % capacity == front` incorrectly:** Forgetting modulo when checking full condition.
- **Not wrapping `rear` and `front` correctly:** Using `rear++` instead of `rear = (rear + 1) % capacity`.
- **Confusing the two strategies:** Mixing the one-empty-slot strategy with the size-tracking strategy.
- **Incorrect size calculation:** `rear - front` gives negative values when wrapped; must use modulo.
- **Not checking empty before dequeue/front:** Causes reading garbage or segfault.

14. Edge Cases

- Empty queue: dequeue, front, rear should all fail gracefully.
- Full queue: enqueue should report overflow.
- Single element: enqueue then dequeue → queue is empty again.
- Wrap-around: after many operations, both pointers wrap correctly.
- Capacity = 1: the queue can hold 0 elements (since one slot is always empty).
- Alternating enqueue/dequeue: should work correctly without overflow.

15. Variations

Variation	What Changes	When Used	Importance
Size-tracked Circular Queue	Use explicit <code>size</code> to track fullness	All slots are usable	Moderate
Overwrite Circular Queue	New elements overwrite oldest when full	Streaming data, real-time logs	Moderate
Thread-safe Circular Queue	Uses locks/atomics for concurrent access	Multithreaded producer-consumer	High — system design

Variation	What Changes	When Used	Importance
Lock-free Circular Queue	Uses atomic operations, no locks	High-performance concurrent systems	Advanced

16. Related Algorithms / Data Structures

Structure	Relationship	How to Choose
Linear Queue	Simpler but wastes space	Use linear for small/unbounded queues
Deque	Can insert/delete at both ends	Use deque when you need both ends
Linked List Queue	Dynamic size, no wrap-around	Use when size is unpredictable
Ring Buffer	Same as circular queue	Systems programming term

17. Practice Problems

Easy

- **Design Circular Queue** — LeetCode (622) — Implement a circular queue. Easy.
- **Moving Average from Data Stream** — LeetCode (346) — Sliding average using circular queue. Easy.

Medium

- **Design Circular Deque** — LeetCode (641) — Circular queue with both ends. Medium.
- **Task Scheduler** — LeetCode (621) — Queue + greedy simulation. Medium.

Hard

- **Design Hit Counter** — LeetCode (362) — Circular queue for timestamp-based counting. Medium (premium).
- **Data Stream as Disjoint Intervals** — LeetCode (352) — Circular queue + set. Hard.

18. Interview Explanation

"A circular queue connects the last position of the array back to the first, forming a circle. This reuses space that would otherwise be wasted after dequeue operations. I use a fixed-size array with front and rear pointers that wrap around using modulo arithmetic. To distinguish between full and empty, I either keep one slot empty, or track the size explicitly. All operations are $O(1)$. Circular queues are used in ring buffers, OS I/O, and anywhere a fixed-size reusable buffer is needed."

19. Revision Notes

- **Key idea:** Reuse space by wrapping front/rear pointers.
- **Full check:** $(rear + 1) \% capacity == front$ (one-empty-slot) or $size == capacity$.
- **Empty check:** $front == rear$ or $size == 0$.
- **Wrap formula:** $(index + 1) \% capacity$.
- **Size formula:** $(rear - front + capacity) \% capacity$.
- **Operations:** All $O(1)$.

- **Trap:** One slot is always empty if using the pointer-only strategy.

20. Final Cheat Sheet

CIRCULAR QUEUE

When to use: Fixed-size buffer, reuse space, ring buffer

Main ops: enqueue(x), dequeue(), isFull(), isEmpty()

Complexity: $O(1)$ each op, $O(\text{capacity})$ space

Key code: rear = (rear+1)%cap; front = (front+1)%cap;
full: (rear+1)%cap == front

Edge cases: empty queue, full queue, wrap-around

Trap: Usable capacity = array_size - 1 (one-empty)

3. Deque (Double-Ended Queue)

1. Overview

A **Deque** (pronounced "deck" — short for **Double-Ended Queue**) is a linear data structure that allows insertion and deletion from **both ends** — front and back. It combines the capabilities of both a stack and a queue.

In C++, `std::deque` is available in the STL. In Python, `collections.deque` is available.

2. Intuition

Think of a line at a buffet where people can join at either end, and people can leave from either end. This is more flexible than a regular queue (only one end for joining) or a stack (only one end for both).

- **Stack operations:** push_front/pop_front OR push_back/pop_back.
- **Queue operations:** push_back + pop_front (FIFO).
- **Deque-specific:** Both ends are equally accessible.

The key insight: **you can add/remove from either end in $O(1)$** . This makes deques extremely versatile.

3. When to Use It

- When you need to **add/remove from both ends** efficiently.
- When you need to implement a **stack + queue** in one structure.
- When you need to **maintain a monotonic sequence** (monotonic deque).
- **Sliding window** problems (maximum/minimum in a window).
- **Palindrome checking** — compare front and back.
- **BFS with two-ended insertion** (0-1 BFS).
- When you need to **reverse a sequence** by popping from both ends.
- When you need **$O(1)$ push/pop at both ends** with good cache locality.

4. When Not to Use It

- When you need **random access** by index (use vector/array).
- When you need **fast insertion in the middle** (use list).
- When you only need a **simple FIFO queue** (use queue — simpler).
- When you only need a **simple LIFO stack** (use stack — simpler).
- When you need **constant-time size** (C++ deque has it, but Python's deque also has it).
- When memory overhead is a concern (deque uses more memory than vector/queue).

5. Core Concepts

Concept	Explanation	Why It Matters
push_front	Insert element at the front	O(1) operation
push_back	Insert element at the back	O(1) operation
pop_front	Remove element from the front	O(1) operation
pop_back	Remove element from the back	O(1) operation
front	Access first element	O(1) peek
back	Access last element	O(1) peek
Monotonic Deque	Deque where elements are always in sorted order (increasing or decreasing)	Used for sliding window min/max

6. Step-by-Step Algorithm

Basic Operations:

1. **Initialize** an empty deque.
2. **push_front(x)**: Add x to the front.
3. **push_back(x)**: Add x to the back.
4. **pop_front()**: Remove and return the front element.
5. **pop_back()**: Remove and return the back element.
6. **front()**: Return the front element without removing it.
7. **back()**: Return the back element without removing it.
8. **empty()**: Check if the deque has no elements.

7. Dry Run

Operations: push_back(1), push_back(2), push_front(0), push_back(3), pop_front(), pop_back(), push_front(-1)

Step	Operation	Deque (front → back)	Size
1	Init	[]	0
2	push_back(1)	[1]	1
3	push_back(2)	[1, 2]	2

Step	Operation	Deque (front → back)	Size
4	push_front(0)	[0, 1, 2]	3
5	push_back(3)	[0, 1, 2, 3]	4
6	pop_front() → 0	[1, 2, 3]	3
7	pop_back() → 3	[1, 2]	2
8	push_front(-1)	[-1, 1, 2]	3

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Deque implementation using a circular array
class Deque {
private:
    int *arr;
    int front, rear, capacity, size;

public:
    Deque(int cap = 100000) {
        capacity = cap;
        arr = new int[capacity];
        front = 0;
        rear = 0;
        size = 0;
    }

    ~Deque() {
        delete[] arr;
    }

    bool isEmpty() {
        return size == 0;
    }

    bool isFull() {
        return size == capacity;
    }

    // Insert at front
    void pushFront(int x) {
        if (isFull()) {
            cout << "Deque is Full\n";
            return;
        }
        front = (front - 1 + capacity) % capacity;
        arr[front] = x;
        size++;
    }
};
```

```
}

// Insert at back
void pushBack(int x) {
    if (isFull()) {
        cout << "Deque is Full\n";
        return;
    }
    arr[rear] = x;
    rear = (rear + 1) % capacity;
    size++;
}

// Remove from front
int popFront() {
    if (isEmpty()) {
        cout << "Deque is Empty\n";
        return -1;
    }
    int val = arr[front];
    front = (front + 1) % capacity;
    size--;
    return val;
}

// Remove from back
int popBack() {
    if (isEmpty()) {
        cout << "Deque is Empty\n";
        return -1;
    }
    rear = (rear - 1 + capacity) % capacity;
    int val = arr[rear];
    size--;
    return val;
}

int getFront() {
    if (isEmpty()) return -1;
    return arr[front];
}

int getBack() {
    if (isEmpty()) return -1;
    return arr[(rear - 1 + capacity) % capacity];
}

int getSize() {
    return size;
}
};

// --- Example Usage ---
int main() {
```

```

Deque dq(10);

dq.pushBack(10);
dq.pushBack(20);
dq.pushFront(5);
dq.pushFront(1);

cout << "Front: " << dq.getFront() << "\n"; // 1
cout << "Back: " << dq.getBack() << "\n"; // 20

cout << "PopFront: " << dq.popFront() << "\n"; // 1
cout << "PopBack: " << dq.popBack() << "\n"; // 20

cout << "Front: " << dq.getFront() << "\n"; // 5
cout << "Back: " << dq.getBack() << "\n"; // 10

return 0;
}

```

9. Python Implementation

```

from collections import deque

# Python's collections.deque is highly optimized
# Example usage:
if __name__ == "__main__":
    dq = deque()

    dq.append(10)      # push_back
    dq.append(20)      # push_back
    dq.appendleft(5)   # push_front
    dq.appendleft(1)   # push_front

    print("Front:", dq[0])      # 1
    print("Back:", dq[-1])     # 20

    print("PopFront:", dq.popleft()) # 1
    print("PopBack:", dq.pop())     # 20

    print("Front:", dq[0])      # 5
    print("Back:", dq[-1])     # 10

```

10. Code Explanation

- **Circular array implementation:** The deque uses a circular array with **front** and **rear** pointers. **front** points to the first element, **rear** points to the next free slot at the back.
- **pushFront:** Decrement **front** (with wrap) and place the element there.
- **pushBack:** Place element at **rear** and increment **rear** (with wrap).
- **popFront:** Read element at **front**, increment **front** (with wrap).

- **popBack:** Decrement **rear** (with wrap) and read the element.
- **pushFront vs pushBack:** Notice that **pushFront** decrements front before writing, while **pushBack** writes at rear and then increments. This convention keeps the deque consistent.
- **Python's deque:** It is implemented as a doubly-linked list of blocks (array of arrays). All operations are $O(1)$. It is the preferred way to use a deque in Python.

11. Complexity Analysis

Operation	Time Complexity	Space Complexity
pushFront	$O(1)$	$O(1)$
pushBack	$O(1)$	$O(1)$
popFront	$O(1)$	$O(1)$
popBack	$O(1)$	$O(1)$
front / back	$O(1)$	$O(1)$
isEmpty / size	$O(1)$	$O(1)$
Overall	$O(1)$ per op	$O(n)$

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Monotonic Deque	"Sliding window max/min", "next greater element"	Maintain deque in increasing/decreasing order	Sliding Window Maximum
0-1 BFS	Graph with 0/1 edge weights	Use deque; push 0-weight edges at front, 1-weight at back	Shortest Path in Binary Matrix
Palindrome Check	"Check if sequence is palindrome"	Compare front and back repeatedly	Valid Palindrome (with deque)
Stack + Queue	Need both LIFO and FIFO	Deque covers both	Undo/Redo buffer
Sliding Window	Fixed window over array	Deque holds window indices	Max of all subarrays of size k

13. Common Mistakes

- Using **std::deque** when **std::queue** or **std::stack** suffices — unnecessarily complex.
- Forgetting that **push_front** decrements, **push_back** increments — leads to off-by-one in custom implementation.
- Not using **collections.deque** in Python — using **list** for pop(0) is $O(n)$.
- Modifying deque while iterating — causes undefined behavior.
- Confusing front and back — especially in monotonic deque problems.
- Not checking for empty deque before accessing front/back.

- **Memory overhead** — `std::deque` is not a single contiguous array; pointer chasing can be slower than vector.

14. Edge Cases

- Empty deque: `popFront`, `popBack`, `front`, `back` should all fail.
- Single element: `popFront` and `popBack` should both work and leave deque empty.
- Alternating `pushFront`/`pushBack`: should work correctly.
- Large number of operations: wrap-around should work correctly.
- `pushFront` on full deque: should report overflow.
- `pushBack` on full deque: should report overflow.

15. Variations

Variation	What Changes	When Used	Importance
Monotonic Deque	Elements are always sorted (increasing/decreasing)	Sliding window min/max, next greater element	Very High
Input-Restricted Deque	Insert at one end only, delete from both	Specialized buffer	Low
Output-Restricted Deque	Delete from one end only, insert at both	Specialized buffer	Low
Block-based Deque	Deque implemented as blocks of arrays (like C++ <code>std::deque</code>)	Better cache locality than linked list	Moderate

16. Related Algorithms / Data Structures

Structure	Relationship	How to Choose
Queue	Deque can act as a queue	Use queue for simpler FIFO-only needs
Stack	Deque can act as a stack	Use stack for simpler LIFO-only needs
Priority Queue	Both are used for scheduling	Deque is not priority-based; use priority queue for priority
Vector/Array	Vector has $O(n)$ front insertion	Use deque for $O(1)$ front/back ops
List (linked)	Both allow $O(1)$ insertion at ends	Deque has better cache locality

17. Practice Problems

Easy

- **Design Circular Deque** — LeetCode (641) — Implement deque using circular array. Easy.
- **Palindrome Linked List** — LeetCode (234) — Can use deque to compare ends. Easy.

Medium

- **Sliding Window Maximum** — LeetCode (239) — Monotonic deque. Medium.
- **Shortest Path in Binary Matrix** — LeetCode (1091) — 0-1 BFS with deque. Medium.

Hard

- **Constrained Subsequence Sum** — LeetCode (1425) — DP + monotonic deque. Hard.
- **Minimum Time to Make Array Sum Zero** — Deque + greedy. Hard.

18. Interview Explanation

"A deque is a double-ended queue that allows $O(1)$ insertion and deletion at both ends. It's more flexible than a queue or stack — it can serve as either. The key application is the monotonic deque, where we maintain elements in sorted order while sliding a window over an array. This gives us $O(n)$ solutions for sliding window maximum/minimum problems. Python's `collections.deque` and C++'s `std::deque` are the standard implementations."

19. Revision Notes

- **Key idea:** $O(1)$ push/pop at both ends.
- **Operations:** `push_front`, `push_back`, `pop_front`, `pop_back`, `front`, `back` — all $O(1)$.
- **Monotonic deque:** maintain sorted order while sliding window.
- **Common use:** sliding window max/min, 0-1 BFS.
- **Python:** `from collections import deque`.
- **C++:** `#include <deque>`.
- **Trap:** Don't use Python list for `pop(0)` — that's $O(n)$.

20. Final Cheat Sheet

DEQUE (Double-Ended Queue)

When to use: Both-end ops, monotonic sliding window, 0-1 BFS

Main ops: `push_front/back`, `pop_front/back`, `front/back`

Complexity: $O(1)$ each op, $O(n)$ space

Key code: `front = (front-1+cap)%cap; arr[front]=x;`
`arr[rear]=x; rear = (rear+1)%cap;`

Python: `from collections import deque`

C++ STL: `#include <deque>`

Trap: Don't use Python list for queue operations

4. BFS Using Queue

1. Overview

Breadth-First Search (BFS) is a graph/tree traversal algorithm that explores all vertices at the current depth level before moving to the next level. It uses a **queue** to maintain the order of exploration.

BFS is the fundamental algorithm for finding the **shortest path** in an unweighted graph, and for **level-order traversal** of trees.

2. Intuition

Imagine you're in a building and want to find a specific room. BFS is like checking all rooms on the current floor before taking the stairs to the next floor. You use a queue to remember which rooms to check next.

- You start at the entrance (source node).
- You check all rooms connected to the entrance (level 1).
- Then all rooms connected to those (level 2), and so on.
- The queue ensures you process nodes in the order they are discovered.

The key insight: **BFS visits nodes in order of their distance from the source**. The first time you reach a node, it's via the shortest path (in terms of number of edges).

3. When to Use It

- Finding the **shortest path** in an unweighted graph.
- **Level-order traversal** of a tree.
- Checking if a graph is **bipartite**.
- Finding **connected components** in an undirected graph.
- **Word Ladder** problems (minimum transformations).
- **Maze shortest path** problems.
- **Flood fill / island perimeter** problems.
- Problems that ask "minimum number of steps/moves" where each move has equal cost.
- Problems with "distance from source" in a grid.
- **Topological sort** using Kahn's algorithm (BFS-based).

4. When Not to Use It

- When the graph has **weighted edges** (use Dijkstra or Bellman-Ford).
- When you need to explore **all paths** (use DFS + backtracking).
- When the graph is **very deep but narrow** (DFS uses less memory).
- When you need to find **if a path exists** between two nodes (DFS is simpler for this).
- When the graph is **infinite or very large** — BFS memory usage grows exponentially.
- When you need **post-order processing** (DFS is natural for this).

5. Core Concepts

Concept	Explanation	Why It Matters
Level	Distance from the source node	BFS processes nodes level by level
Visited Array	Marks nodes that have been explored	Prevents infinite loops and revisits

Concept	Explanation	Why It Matters
Queue	Holds nodes to be processed next	Ensures FIFO order = level-by-level
Distance Array	Stores shortest distance from source	BFS guarantees shortest path in unweighted graphs
Parent Array	Stores the previous node in the path	Used to reconstruct the shortest path
Adjacency List	Representation of the graph	Efficient for BFS traversal

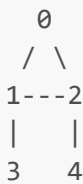
6. Step-by-Step Algorithm

BFS on a graph (from source node **s**):

1. Create a visited array (bool), initialized to **false** for all nodes.
2. Create a distance array (int), initialized to **INF** for all nodes.
3. Create a queue.
4. Mark **s** as visited. Set **dist[s] = 0**. Push **s** into the queue.
5. While the queue is not empty: a. Pop the front node **u**. b. For each neighbor **v** of **u**:
 - If **v** is not visited:
 - Mark **v** as visited.
 - Set **dist[v] = dist[u] + 1**.
 - Push **v** into the queue.
6. After the loop, **dist** contains shortest distances from **s** to all reachable nodes.

7. Dry Run

Graph: 5 nodes (0–4). Edges: 0-1, 0-2, 1-3, 2-4. Source = 0.



Step	Queue (front → back)	Node Popped	Visited	Distance (dist[])
Init	[0]	—	{0}	[0, ∞, ∞, ∞, ∞]
1	[]	0	{0}	[0, ∞, ∞, ∞, ∞]
2	[1, 2]	—	{0, 1, 2}	[0, 1, 1, ∞, ∞]
3	[2]	1	{0, 1, 2}	[0, 1, 1, ∞, ∞]
4	[2, 3]	—	{0, 1, 2, 3}	[0, 1, 1, 2, ∞]
5	[3]	2	—	—

Step	Queue (front → back)	Node Popped	Visited	Distance (dist[])
6	[3, 4]	—	{0, 1, 2, 3, 4}	[0, 1, 1, 2, 2]
7	[4]	3	—	—
8	[]	4	—	—

Final distances: dist[0]=0, dist[1]=1, dist[2]=1, dist[3]=2, dist[4]=2.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// BFS from a single source
vector<int> bfs(int n, vector<int> adj[], int source) {
    vector<int> dist(n, INT_MAX);
    vector<bool> visited(n, false);
    queue<int> q;

    // Initialize source
    visited[source] = true;
    dist[source] = 0;
    q.push(source);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }

    return dist;
}

// BFS returning level-order traversal (for trees)
vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> result;
    if (!root) return result;

    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        int levelSize = q.size();
        vector<int> currentLevel;
```

```

        for (int i = 0; i < levelSize; i++) {
            TreeNode* node = q.front();
            q.pop();
            currentLevel.push_back(node->val);

            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }

        result.push_back(currentLevel);
    }

    return result;
}

// BFS in a grid (4-directional)
int bfsGrid(vector<vector<int>>& grid, pair<int,int> start, pair<int,int> target)
{
    int n = grid.size(), m = grid[0].size();
    vector<vector<int>> dist(n, vector<int>(m, -1));
    queue<pair<int,int>> q;

    int dx[] = {-1, 1, 0, 0};
    int dy[] = {0, 0, -1, 1};

    dist[start.first][start.second] = 0;
    q.push(start);

    while (!q.empty()) {
        auto [x, y] = q.front();
        q.pop();

        if (x == target.first && y == target.second) {
            return dist[x][y];
        }

        for (int d = 0; d < 4; d++) {
            int nx = x + dx[d];
            int ny = y + dy[d];

            if (nx >= 0 && nx < n && ny >= 0 && ny < m
                && grid[nx][ny] != 0 && dist[nx][ny] == -1) {
                dist[nx][ny] = dist[x][y] + 1;
                q.push({nx, ny});
            }
        }
    }

    return -1; // unreachable
}

// --- Example Usage ---
int main() {

```

```
// Graph: 5 nodes, edges as adjacency list
int n = 5;
vector<int> adj[n];
adj[0] = {1, 2};
adj[1] = {0, 2, 3};
adj[2] = {0, 1, 4};
adj[3] = {1};
adj[4] = {2};

vector<int> dist = bfs(n, adj, 0);

for (int i = 0; i < n; i++) {
    cout << "Dist to " << i << ": " << dist[i] << "\n";
}

return 0;
}
```

9. Python Implementation

```
from collections import deque

def bfs(n, adj, source):
    """BFS from a single source. Returns distance array."""
    dist = [float('inf')] * n
    visited = [False] * n
    q = deque()

    visited[source] = True
    dist[source] = 0
    q.append(source)

    while q:
        u = q.popleft()
        for v in adj[u]:
            if not visited[v]:
                visited[v] = True
                dist[v] = dist[u] + 1
                q.append(v)

    return dist

def level_order(root):
    """Level-order traversal of a binary tree."""
    if not root:
        return []

    result = []
    q = deque([root])
```

```

while q:
    level_size = len(q)
    current_level = []

    for _ in range(level_size):
        node = q.popleft()
        current_level.append(node.val)
        if node.left:
            q.append(node.left)
        if node.right:
            q.append(node.right)

    result.append(current_level)

return result

def bfs_grid(grid, start, target):
    """BFS on a grid. 0 = blocked, 1 = open."""
    n, m = len(grid), len(grid[0])
    dist = [[-1] * m for _ in range(n)]
    q = deque()

    dx = [-1, 1, 0, 0]
    dy = [0, 0, -1, 1]

    sx, sy = start
    dist[sx][sy] = 0
    q.append((sx, sy))

    while q:
        x, y = q.popleft()

        if (x, y) == target:
            return dist[x][y]

        for d in range(4):
            nx, ny = x + dx[d], y + dy[d]

            if 0 <= nx < n and 0 <= ny < m and grid[nx][ny] != 0 and dist[nx][ny]
== -1:
                dist[nx][ny] = dist[x][y] + 1
                q.append((nx, ny))

    return -1 # unreachable

# Example usage
if __name__ == "__main__":
    n = 5
    adj = [
        [1, 2],
        [0, 2, 3],

```

```

    [0, 1, 4],
    [1],
    [2]
]
dist = bfs(n, adj, 0)
for i, d in enumerate(dist):
    print(f"Dist to {i}: {d}")

```

10. Code Explanation

- **Visited array:** Prevents re-visiting nodes. Without it, we could get infinite loops in cyclic graphs.
- **Distance array:** Stores the shortest distance from source. BFS guarantees this is the minimum because we process nodes in order of increasing distance.
- **Queue:** The core data structure. We pop from the front (FIFO) to ensure level-by-level processing.
- **Neighbor iteration:** For each node, we look at all its neighbors. In a graph, this is the adjacency list. In a grid, it's the 4 or 8 directional neighbors.
- **Level-order (tree):** We process nodes level by level. The inner loop runs `levelSize` times, which is the number of nodes at the current level. This allows us to group nodes by level.
- **Grid BFS:** We use coordinate pairs and check boundaries. The distance is incremented by 1 for each step.

11. Complexity Analysis

Aspect	Complexity
Time	$O(V + E)$ — each node and edge is processed once
Space (adjacency list)	$O(V + E)$ for the graph
Space (queue)	$O(V)$ — worst case, all nodes in queue
Space (visited)	$O(V)$
Space (distance)	$O(V)$
Grid BFS Time	$O(N \times M)$ — each cell visited once
Grid BFS Space	$O(N \times M)$ — distance array

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Shortest Path in Unweighted Graph	"Minimum number of edges", "shortest path"	BFS from source	Word Ladder, Snakes and Ladders
Level-Order Traversal	"Print level by level", "zigzag"	BFS with level size tracking	Binary Tree Level Order Traversal
Multi-Source BFS	"Start from multiple points", "minimum distance to nearest 0"	Push all sources initially	01 Matrix, Rotting Oranges

Pattern	How to Identify	Approach	Example Problems
BFS on Grid	"Maze", "grid", "shortest path in matrix"	BFS with direction arrays	Shortest Path in Binary Matrix
BFS for Bipartite Check	"Can we color graph with 2 colors"	BFS, assign alternating colors	Is Graph Bipartite?
Kahn's Algorithm (Topological Sort)	"Ordering of tasks", "course schedule"	BFS with in-degree array	Course Schedule II

13. Common Mistakes

- **Not marking visited when pushing to queue** — marking only when popping causes re-queuing and infinite loops.
- **Using DFS instead of BFS** for shortest path in unweighted graphs — DFS doesn't guarantee shortest path.
- **Not resetting visited array** between multiple BFS calls.
- **Forgetting to handle disconnected graphs** — BFS from a single source only visits reachable nodes.
- **Infinite loop in cyclic graphs** — visited array prevents this.
- **Not checking bounds in grid BFS** — causes out-of-bounds errors.
- **Using recursion for BFS** — BFS is inherently iterative; recursion is for DFS.
- **Confusing row and column indices** in grid BFS.

14. Edge Cases

- Single node: BFS should just process the source.
- Disconnected graph: unreachable nodes should have INF distance.
- Empty graph: BFS should handle gracefully.
- Graph with cycles: visited array prevents infinite loops.
- Grid with all blocked cells: BFS should return -1 (unreachable).
- Start == target: BFS should return 0 immediately.
- Large grid: ensure $O(N \times M)$ time and space are acceptable.
- Directed graph: BFS only follows edges in the direction they are defined.

15. Variations

Variation	What Changes	When Used	Importance
Multi-Source BFS	Push multiple sources into queue initially	Minimum distance to nearest source	Very High
Bidirectional BFS	BFS from both source and target simultaneously	Faster shortest path in large graphs	High (CP)
0-1 BFS	Deque instead of queue for 0/1 weight edges	Shortest path with 0/1 weights	High
BFS with State	Queue holds (node, state) for problems with extra state	Word Ladder, puzzles	Moderate

Variation	What Changes	When Used	Importance
BFS with Bitmask	Queue holds (node, bitmask) for visited state	Travelling salesman (small n)	Advanced

16. Related Algorithms / Data Structures

Structure	Relationship	How to Choose
DFS	Both traverse graphs	BFS for shortest path; DFS for path existence, cycles, topological sort
Dijkstra	BFS for weighted graphs	BFS for unweighted; Dijkstra for weighted
0-1 BFS	BFS for graphs with 0/1 weights	Use deque instead of queue
Topological Sort (Kahn's)	BFS-based	Use BFS with in-degree tracking
Union-Find (DSU)	Both find connectivity	BFS for distances; DSU for connectivity queries

17. Practice Problems

Easy

- **Binary Tree Level Order Traversal** — LeetCode (102) — BFS level-order. Easy.
- **N-ary Tree Level Order Traversal** — LeetCode (429) — Same pattern. Easy.

Medium

- **Word Ladder** — LeetCode (127) — BFS on implicit graph (words). Medium.
- **Shortest Path in Binary Matrix** — LeetCode (1091) — Grid BFS. Medium.

Hard

- **Sliding Puzzle** — LeetCode (773) — BFS with state. Hard.
- **Minimum Number of Flips to Convert Binary Matrix** — LeetCode (1284) — BFS with bitmask. Hard.

18. Interview Explanation

"BFS is a graph traversal algorithm that explores nodes level by level. I use a queue to maintain the order — when I pop a node, I push all its unvisited neighbors. This guarantees that the first time I reach a node, it's via the shortest path in terms of number of edges. The time complexity is $O(V+E)$ and space is $O(V)$. BFS is used for shortest paths in unweighted graphs, level-order tree traversal, and multi-source problems like rotting oranges. The key implementation detail is to mark nodes as visited when pushing them to the queue, not when popping."

19. Revision Notes

- **Key idea:** Level-by-level traversal using a queue.

- **Shortest path guarantee:** BFS finds shortest path in unweighted graphs.
- **Visited:** Mark when pushing, not when popping.
- **Time:** $O(V+E)$, Space: $O(V)$.
- **Multi-source BFS:** Push all sources initially.
- **Grid BFS:** Use direction arrays, check bounds.
- **Common trap:** Forgetting visited array $\rightarrow$ infinite loop in cyclic graphs.

20. Final Cheat Sheet

BFS (Breadth-First Search)

When to use: Shortest path (unweighted), level-order, multi-source distance, connected components

Data structure: Queue

Complexity: $O(V+E)$ time, $O(V)$ space

Key code:

```
q.push(src); visited[src]=true;
while(!q.empty()) { u=q.front(); q.pop();
for(v: adj[u]) if(!visited[v]) { ... } }
```

Edge cases: Disconnected graph, cycles, single node

Trap: Mark visited when pushing to queue

5. Sliding Window Maximum

1. Overview

Sliding Window Maximum is a technique to find the maximum element in every contiguous subarray of size k in an array. The naive $O(n \cdot k)$ approach checks each window separately, but the optimal solution uses a **monotonic deque** to achieve $O(n)$.

This is one of the most important deque problems for placements.

2. Intuition

Imagine you're looking through a window of size k that slides over an array. At each position, you need to know the maximum element visible.

The naive approach: each time the window moves, scan all k elements $\rightarrow O(n \cdot k)$.

The deque approach: maintain a deque of **indices** where the elements are in **decreasing order**. The front of the deque is always the maximum for the current window.

When the window slides:

- Remove the element that just left the window (if it's at the front of the deque).

- Remove elements from the back of the deque that are smaller than the new element (they can never be the maximum).
- Add the new element at the back.

The key insight: **if a smaller element comes after a larger element, the smaller element can only become the maximum after the larger one leaves the window**. So we can discard smaller elements from the back of the deque.

3. When to Use It

- When the problem asks for "**maximum/minimum in every window of size k**".
- When the array is **large** and $O(n \cdot k)$ is too slow.
- When you see "**subarray of size k**" + "**maximum/minimum**".
- When you need to compute **sliding window statistics** efficiently.
- When the problem is about "**maximum of all subarrays of size k**".

4. When Not to Use It

- When **k** is very small (like 1 or 2) — naive $O(n \cdot k)$ is fine.
- When you need **other window statistics** like sum, average, median — use different techniques (prefix sum, two heaps for median).
- When the array is **dynamic** (insertions/deletions) — use a segment tree.
- When you need **all subarrays** (not just fixed size k) — use stack-based approach for next greater element.
- When **k** is large and close to **n** — the deque is still $O(n)$, but the overhead may not be worth it.

5. Core Concepts

Concept	Explanation	Why It Matters
Monotonic Decreasing Deque	Deque where elements are in decreasing order (largest at front)	Front is always the maximum of current window
Indices vs Values	Store indices in the deque, not values	Allows checking if an element is still in the window
Window Sliding	Remove leftmost element (if out of window), add new element	Maintains the window invariant
Backward Cleanup	Remove smaller elements from back before adding new element	Maintains decreasing order

6. Step-by-Step Algorithm

Problem: Given array `arr[0..n-1]` and window size **k**, find the maximum in each window.

1. Initialize an empty deque `dq` (will store indices).
2. Initialize an empty result array.
3. For the first **k** elements ($i = 0$ to $k-1$):
 - While `dq` is not empty and `arr[dq.back()] <= arr[i]`:

- Pop from back.
 - Push `i` to the back of `dq`.
- 4. Add `arr[dq.front()]` to result (max of first window).
- 5. For `i = k` to `n-1`:
 - **Remove out-of-window elements:** While `dq` is not empty and `dq.front() <= i - k`:
 - Pop from front.
 - **Maintain decreasing order:** While `dq` is not empty and `arr[dq.back()] <= arr[i]`:
 - Pop from back.
 - **Add new element:** Push `i` to the back of `dq`.
 - **Record max:** Add `arr[dq.front()]` to result.
- 6. Return result.

7. Dry Run

Array: [1, 3, -1, -3, 5, 3, 6, 7], **k = 3**

i	Element	Deque (indices)	Deque (values)	Window Max
0	1	[0]	[1]	—
1	3	[1]	[3]	—
2	-1	[1, 2]	[3, -1]	3
3	-3	[1, 2, 3]	[3, -1, -3]	3
4	5	[4]	[5]	5
5	3	[4, 5]	[5, 3]	5
6	6	[6]	[6]	6
7	7	[7]	[7]	7

Step-by-step:

- **i=0:** `dq=[0]`, `arr[0]=1`. No smaller elements to remove.
- **i=1:** `arr[1]=3 > arr[0]=1`, so pop 0. `dq=[1]`.
- **i=2:** `arr[2]=-1`. No smaller elements at back. `dq=[1,2]`. First window: `max = arr[1] = 3`.
- **i=3:** `arr[3]=-3`. `dq=[1,2,3]`. Window `max = arr[1] = 3`. (indices 1,2,3 are in window [1..3])
- **i=4:** Remove front if out of window: `front=1`, `i-k = 4-3=1`, so 1 is not `> 1`, keep it. Actually `1 <= 1`, so pop front. `dq=[2,3]`. Now `arr[4]=5 > arr[3]=-3`, pop 3. `arr[4]=5 > arr[2]=-1`, pop 2. `dq=[4]`. `max = 5`.
- **i=5:** `arr[5]=3`. `dq=[4,5]`. `max = arr[4] = 5`.
- **i=6:** `arr[6]=6 > arr[5]=3`, pop 5. `arr[6]=6 > arr[4]=5`, pop 4. `dq=[6]`. `max = 6`.
- **i=7:** `arr[7]=7 > arr[6]=6`, pop 6. `dq=[7]`. `max = 7`.

Result: [3, 3, 5, 5, 6, 7]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Sliding Window Maximum
vector<int> slidingWindowMax(vector<int>& arr, int k) {
    int n = arr.size();
    if (n == 0 || k == 0) return {};

    deque<int> dq; // stores indices
    vector<int> result;

    // Process first window
    for (int i = 0; i < k; i++) {
        // Remove smaller elements from back
        while (!dq.empty() && arr[dq.back()] <= arr[i]) {
            dq.pop_back();
        }
        dq.push_back(i);
    }
    result.push_back(arr[dq.front()]);

    // Process remaining windows
    for (int i = k; i < n; i++) {
        // Remove elements out of current window
        while (!dq.empty() && dq.front() <= i - k) {
            dq.pop_front();
        }

        // Remove smaller elements from back
        while (!dq.empty() && arr[dq.back()] <= arr[i]) {
            dq.pop_back();
        }

        dq.push_back(i);
        result.push_back(arr[dq.front()]);
    }

    return result;
}

// --- Example Usage ---
int main() {
    vector<int> arr = {1, 3, -1, -3, 5, 3, 6, 7};
    int k = 3;

    vector<int> result = slidingWindowMax(arr, k);

    cout << "Sliding Window Maximums: ";
    for (int x : result) {
        cout << x << " ";
    }
    // Output: 3 3 5 5 6 7
}
```

```
    cout << "\n";

    return 0;
}
```

9. Python Implementation

```
from collections import deque

def sliding_window_max(arr, k):
    """Return maximum in every window of size k."""
    n = len(arr)
    if n == 0 or k == 0:
        return []

    dq = deque() # stores indices
    result = []

    # Process first window
    for i in range(k):
        while dq and arr[dq[-1]] <= arr[i]:
            dq.pop()
        dq.append(i)
    result.append(arr[dq[0]])

    # Process remaining windows
    for i in range(k, n):
        # Remove elements out of current window
        while dq and dq[0] <= i - k:
            dq.popleft()

        # Remove smaller elements from back
        while dq and arr[dq[-1]] <= arr[i]:
            dq.pop()

        dq.append(i)
        result.append(arr[dq[0]])

    return result

# Example usage
if __name__ == "__main__":
    arr = [1, 3, -1, -3, 5, 3, 6, 7]
    k = 3
    result = sliding_window_max(arr, k)
    print("Sliding Window Maximums:", result)
    # Output: [3, 3, 5, 5, 6, 7]
```

10. Code Explanation

- **Deque of indices:** We store indices in the deque, not values. This allows us to check if an element is still within the current window by comparing `dq.front()` with `i - k`.
- **First window (lines 7-11):** We build the initial monotonic deque for the first `k` elements. The front of the deque is the maximum of the first window.
- **Maintaining decreasing order:** When a new element arrives, we pop from the back all elements that are $\leq$ the new element. These elements can never be the maximum for any future window because the new element is larger and will stay in the window longer.
- **Removing out-of-window elements:** Before recording the max for each window, we pop from the front any index that is $\leq i - k$ (i.e., outside the current window).
- **Recording the max:** The front of the deque is always the maximum of the current window.

11. Complexity Analysis

Aspect	Complexity
Time	$O(n)$ — each element is pushed and popped at most once
Space	$O(k)$ — deque stores at most <code>k</code> elements
Best case	$O(n)$ — already decreasing array
Worst case	$O(n)$ — already increasing array (each element causes all previous to be popped)

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Sliding Window Maximum	"Max in every window of size <code>k</code> "	Monotonic decreasing deque	Sliding Window Maximum
Sliding Window Minimum	"Min in every window of size <code>k</code> "	Monotonic increasing deque	Sliding Window Minimum
First Negative in Window	"First negative in every window"	Queue of negative indices	First Negative in Every Window of Size <code>K</code>
Sliding Window Median	"Median of every window"	Two heaps + lazy deletion	Sliding Window Median
Count of smaller elements in window	"Count of elements $\leq$ <code>X</code> in window"	Monotonic deque + binary search	Count of Smaller Numbers After Self

13. Common Mistakes

- **Storing values instead of indices** — cannot determine if an element is out of the window.
- **Using `<=` vs `<`** — for maximum, we pop when `arr[back] <= arr[i]`. Using `<` only keeps the first occurrence of a duplicate, which is fine but may keep unnecessary elements.
- **Not checking if deque is empty** before accessing `front()` or `back()`.

- **Forgetting to process the first window separately** — the first window's max is recorded before the loop.
- **Incorrect out-of-window condition** — using $< i-k$ instead of $\leq i-k$. The element at index $i-k$ has just left the window.
- **Using queue instead of deque** — need to pop from both ends.
- **Not handling $k > n$ case** — should return empty array or handle gracefully.

14. Edge Cases

- Empty array: return empty.
- $k = 1$: each element is its own window max $\rightarrow$ return the array itself.
- $k = n$: return the single maximum of the whole array.
- $k > n$: return empty or handle gracefully.
- All equal elements: deque will have exactly one element (since we pop equals).
- Increasing array: deque always has one element (the newest one).
- Decreasing array: deque has all k elements.
- Single element: works with $k=1$.
- Negative numbers: algorithm works for any integers.

15. Variations

Variation	What Changes	When Used	Importance
Sliding Window Minimum	Use increasing deque instead of decreasing	Finding minimum in every window	High
Sliding Window Max in 2D	Apply sliding window max on rows, then columns	Maximum in every $k \times k$ submatrix	Moderate
Sliding Window Max with Deque of Values	Use <code>pair<value, index></code> instead of indices	Alternative implementation	Low
Sliding Window Max — Circular Array	Handle wrap-around for circular arrays	Circular window problems	Moderate

16. Related Algorithms / Data Structures

Structure	Relationship	How to Choose
Monotonic Stack	Same idea but for non-sliding window	Use stack for next greater element; deque for sliding window
Segment Tree	Can answer range max queries	Use deque for $O(n)$ sliding window; segment tree for arbitrary ranges
Priority Queue (Max-Heap)	Can get max but $O(n \log k)$	Use deque for $O(n)$ sliding window max
Sparse Table	Range max queries in $O(1)$	Use sparse table for static array; deque for sliding window

17. Practice Problems

Easy

- **Maximum Average Subarray I** — LeetCode (643) — Simple sliding window sum. Easy.
- **Contains Duplicate II** — LeetCode (219) — Sliding window with set. Easy.

Medium

- **Sliding Window Maximum** — LeetCode (239) — The classic problem. Medium.
- **Minimum Size Subarray Sum** — LeetCode (209) — Two-pointer sliding window. Medium.

Hard

- **Sliding Window Median** — LeetCode (480) — Two heaps + lazy deletion. Hard.
- **Minimum Number of Flips to Make the Binary String Alternating** — LeetCode (1888) — Sliding window with deque. Hard.

18. Interview Explanation

"Sliding window maximum uses a monotonic deque to find the maximum in every window of size k in $O(n)$ time. The key insight is that we maintain a deque of indices in decreasing order of their values. The front is always the maximum of the current window. When the window slides, we remove the element that left the window from the front, and remove from the back all elements $\leq$ the new element — they can never be the maximum. This works because each element is added and removed at most once, giving $O(n)$ total time."

19. Revision Notes

- **Key idea:** Monotonic decreasing deque of indices.
- **Front:** always the maximum of current window.
- **Back:** remove elements $\leq$ new element before adding.
- **Front:** remove elements out of window (index $\leq i-k$).
- **Time:** $O(n)$, Space: $O(k)$.
- **Store indices, not values** — need to check if element is still in window.
- **Common trap:** Off-by-one in out-of-window condition (use $\leq i-k$).

20. Final Cheat Sheet

SLIDING WINDOW MAXIMUM

```
When to use: Max/min in every window of size k
Data structure: Monotonic deque (indices, not values)
Complexity:    O(n) time, O(k) space
Key code:      while(!dq.empty() && arr[dq.back()]<=arr[i])
                dq.pop_back();
                dq.push_back(i);
                while(!dq.empty() && dq.front()<=i-k)
```

```
        dq.pop_front();  
Result:    arr[dq.front()] is max of window  
Edge cases: k=1 (return array), k=n (single max)  
Trap:      Store indices, not values
```

6. Monotonic Deque

1. Overview

A **Monotonic Deque** is a deque that maintains its elements in a strictly increasing or strictly decreasing order at all times. It is a **pattern** rather than a data structure in itself — it's a way of using a deque to solve problems efficiently.

The monotonic deque pattern is the foundation for solving sliding window min/max, next greater/smaller element, and many other problems in $O(n)$ time.

2. Intuition

Think of a line of people standing in decreasing order of height. The tallest person is at the front. When a new person arrives:

- If they are taller than the person at the back, the shorter person at the back leaves (they can never be the tallest in any future window).
- The new person then joins at the back.

This "cleaning up" of the back ensures that the deque always stays sorted. The key insight: **any element that is smaller than a newer element will never be the answer for any future window.**

3. When to Use It

- When you need to find the **next greater element** (NGE) or **next smaller element** (NSE).
- When you need to find the **previous greater element** (PGE) or **previous smaller element** (PSE).
- **Sliding window maximum** or **minimum**.
- When you need to **maintain a running minimum/maximum** in a sliding window.
- When elements have a "**natural ordering**" and you need to compare them with future elements.
- Problems where you need to **discard elements that are "dominated"** by newer elements.

4. When Not to Use It

- When you need **random access** to the results — use a segment tree or sparse table.
- When the array is **static** and you need many queries — use a sparse table for $O(1)$ queries.
- When you need **range queries** on arbitrary intervals (not just sliding window) — use segment tree.
- When the problem can be solved with a **simple two-pointer** approach.
- When the monotonic property doesn't help — not all problems benefit from this pattern.

5. Core Concepts

Concept	Explanation	Why It Matters
Monotonic Increasing	Elements are in strictly increasing order (smallest at front)	Front is minimum of current window
Monotonic Decreasing	Elements are in strictly decreasing order (largest at front)	Front is maximum of current window
Backward Cleanup	Removing elements from the back that violate monotonicity	Maintains the invariant
Forward Cleanup	Removing elements from the front that are out of range	For sliding window problems
Indices vs Values	Usually store indices to check range, compare values	Allows range checking

6. Step-by-Step Algorithm

Monotonic Decreasing Deque (for sliding window max):

1. Initialize empty deque.
2. For each element `arr[i]`:
 - While deque not empty and `arr[deque.back()] <= arr[i]`:
 - Pop back (these elements are dominated by `arr[i]`).
 - Push `i` to back.
 - While deque not empty and `deque.front() < i - k + 1`:
 - Pop front (out of window).
 - If `i >= k - 1`:
 - Record `arr[deque.front()]` as answer.

Monotonic Increasing Deque (for sliding window min):

1. Initialize empty deque.
2. For each element `arr[i]`:
 - While deque not empty and `arr[deque.back()] >= arr[i]`:
 - Pop back.
 - Push `i` to back.
 - While deque not empty and `deque.front() < i - k + 1`:
 - Pop front.
 - If `i >= k - 1`:
 - Record `arr[deque.front()]` as answer.

7. Dry Run

Monotonic Decreasing Deque on [2, 5, 3, 7, 1, 4], k = 3

i	arr[i]	Deque (indices)	Deque (values)	Window	Max
0	2	[0]	[2]	—	—

i	arr[i]	Deque (indices)	Deque (values)	Window	Max
1	5	[1]	[5]	—	—
2	3	[1, 2]	[5, 3]	[0..2]	5
3	7	[3]	[7]	[1..3]	7
4	1	[3, 4]	[7, 1]	[2..4]	7
5	4	[3, 5]	[7, 4]	[3..5]	7

Step-by-step:

- **i=0 (2):** dq=[0]. dq front=0, $i-k+1 = 0-3+1 = -2$, so $0 > -2$, keep.
- **i=1 (5):** Pop 0 ($5 > 2$). dq=[1].
- **i=2 (3):** $3 \leq 5$, keep. dq=[1,2]. Window [0..2]: max = arr[1] = 5.
- **i=3 (7):** Pop 2 ($7 > 3$), pop 1 ($7 > 5$). dq=[3]. Window [1..3]: max = 7. Check front: dq[0]=3, $i-k+1 = 3-3+1=1$, $3 \geq 1$, keep.
- **i=4 (1):** $1 \leq 7$, keep. dq=[3,4]. Window [2..4]: max = 7. Check front: dq[0]=3, $i-k+1 = 4-3+1=2$, $3 \geq 2$, keep.
- **i=5 (4):** Pop 4 ($4 > 1$). dq=[3,5]. Window [3..5]: max = 7. Check front: dq[0]=3, $i-k+1 = 5-3+1=3$, $3 \geq 3$, keep.

Result: [5, 7, 7, 7]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Monotonic Decreasing Deque – Sliding Window Maximum
vector<int> slidingWindowMax(vector<int>& arr, int k) {
    int n = arr.size();
    vector<int> result;
    deque<int> dq; // indices, values in decreasing order

    for (int i = 0; i < n; i++) {
        // Backward cleanup: remove smaller elements from back
        while (!dq.empty() && arr[dq.back()] <= arr[i]) {
            dq.pop_back();
        }
        dq.push_back(i);

        // Forward cleanup: remove elements out of window
        if (dq.front() <= i - k) {
            dq.pop_front();
        }

        // Record result when window is fully formed
        if (i >= k - 1) {
            result.push_back(arr[dq.front()]);
        }
    }
}
```

```

    }
}

return result;
}

// Monotonic Increasing Deque – Sliding Window Minimum
vector<int> slidingWindowMin(vector<int>& arr, int k) {
    int n = arr.size();
    vector<int> result;
    deque<int> dq; // indices, values in increasing order

    for (int i = 0; i < n; i++) {
        // Backward cleanup: remove larger elements from back
        while (!dq.empty() && arr[dq.back()] >= arr[i]) {
            dq.pop_back();
        }
        dq.push_back(i);

        // Forward cleanup: remove elements out of window
        if (dq.front() <= i - k) {
            dq.pop_front();
        }

        // Record result when window is fully formed
        if (i >= k - 1) {
            result.push_back(arr[dq.front()]);
        }
    }

    return result;
}

// Next Greater Element (NGE) – using monotonic stack (same principle)
vector<int> nextGreaterElement(vector<int>& arr) {
    int n = arr.size();
    vector<int> result(n, -1);
    stack<int> st; // monotonic decreasing stack of indices

    for (int i = 0; i < n; i++) {
        while (!st.empty() && arr[st.top()] < arr[i]) {
            result[st.top()] = arr[i];
            st.pop();
        }
        st.push(i);
    }

    return result;
}

// --- Example Usage ---
int main() {
    vector<int> arr = {2, 5, 3, 7, 1, 4};
    int k = 3;

```

```

vector<int> maxResult = slidingWindowMax(arr, k);
cout << "Sliding Window Max: ";
for (int x : maxResult) cout << x << " ";
cout << "\n";

vector<int> minResult = slidingWindowMin(arr, k);
cout << "Sliding Window Min: ";
for (int x : minResult) cout << x << " ";
cout << "\n";

vector<int> nge = nextGreaterElement(arr);
cout << "Next Greater Element: ";
for (int x : nge) cout << x << " ";
cout << "\n";

return 0;
}

```

9. Python Implementation

```

from collections import deque

def sliding_window_max(arr, k):
    """Monotonic decreasing deque for sliding window max."""
    n = len(arr)
    result = []
    dq = deque() # indices, values in decreasing order

    for i in range(n):
        # Backward cleanup: remove smaller elements from back
        while dq and arr[dq[-1]] <= arr[i]:
            dq.pop()
        dq.append(i)

        # Forward cleanup: remove elements out of window
        if dq[0] <= i - k:
            dq.popleft()

        # Record result when window is fully formed
        if i >= k - 1:
            result.append(arr[dq[0]])

    return result

def sliding_window_min(arr, k):
    """Monotonic increasing deque for sliding window min."""
    n = len(arr)
    result = []

```

```

dq = deque() # indices, values in increasing order

for i in range(n):
    # Backward cleanup: remove larger elements from back
    while dq and arr[dq[-1]] >= arr[i]:
        dq.pop()
    dq.append(i)

    # Forward cleanup: remove elements out of window
    if dq[0] <= i - k:
        dq.popleft()

    # Record result when window is fully formed
    if i >= k - 1:
        result.append(arr[dq[0]])

return result

def next_greater_element(arr):
    """Next Greater Element using monotonic stack."""
    n = len(arr)
    result = [-1] * n
    stack = [] # monotonic decreasing stack of indices

    for i in range(n):
        while stack and arr[stack[-1]] < arr[i]:
            result[stack.pop()] = arr[i]
        stack.append(i)

    return result

# Example usage
if __name__ == "__main__":
    arr = [2, 5, 3, 7, 1, 4]
    k = 3

    print("Sliding Window Max:", sliding_window_max(arr, k))
    print("Sliding Window Min:", sliding_window_min(arr, k))
    print("Next Greater Element:", next_greater_element(arr))

```

10. Code Explanation

- **Backward cleanup:** The core of the monotonic deque. When a new element arrives, we remove all elements from the back that are "dominated" — smaller elements (for max) or larger elements (for min). These dominated elements can never be the answer for any future window because the new element is better and will stay in the window longer.
- **Forward cleanup:** For sliding window problems, we remove elements from the front that are no longer in the current window. This is done by checking if the index is $\leq i - k$.

- **Recording results:** We only record results when the window is fully formed ($i \geq k - 1$). The front of the deque is always the maximum (or minimum) of the current window.
- **NGE using stack:** The same monotonic principle applies. We use a stack instead of a deque because we don't need to remove from the front. Elements are waiting for their next greater element on the stack.

11. Complexity Analysis

Aspect	Complexity
Time (Sliding Window)	$O(n)$ — each element pushed and popped at most once
Space (Sliding Window)	$O(k)$ — deque size at most k
Time (NGE)	$O(n)$ — each element pushed and popped at most once
Space (NGE)	$O(n)$ — result array

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Sliding Window Max	"Max in every window of size k "	Monotonic decreasing deque	LeetCode 239
Sliding Window Min	"Min in every window of size k "	Monotonic increasing deque	Sliding Window Minimum
Next Greater Element	"Next larger element to the right"	Monotonic decreasing stack	LeetCode 496
Previous Greater Element	"Larger element to the left"	Iterate left to right with stack	Stock Span Problem
Largest Rectangle in Histogram	"Largest rectangle area"	Monotonic stack for boundaries	LeetCode 84
Trapping Rain Water	"Water trapped between bars"	Two pointers or monotonic stack	LeetCode 42

13. Common Mistakes

- **Using $<$ instead of $<=$ (or vice versa)** — for strict monotonicity vs non-strict. For duplicates, using $<=$ removes the earlier duplicate, which is usually fine.
- **Forgetting forward cleanup** in sliding window problems — the deque will contain stale indices.
- **Not checking empty deque** before accessing front/back.
- **Confusing increasing vs decreasing** — use decreasing for max, increasing for min.
- **Using deque when stack is enough** — for NGE problems, a stack is simpler.
- **Not understanding that the deque stores indices, not values** — values are compared via `arr[index]`.

14. Edge Cases

- Empty array: return empty result.
- $k = 1$: each element is the max/min of its own window.
- $k = n$: single result.
- All equal elements: deque will have one element (since we pop equals).
- Strictly increasing array: for max, deque always has one element.
- Strictly decreasing array: for max, deque has all k elements.
- Duplicates: using $\leq$ vs $<$ affects behavior.

15. Variations

Variation	What Changes	When Used	Importance
Monotonic Stack	Stack instead of deque	NGE, PGE, L-Rectangle	Very High
Monotonic Queue	Queue-only (not deque)	When only front removal is needed	Low
2D Monotonic Deque	Apply on rows then columns	Max in $k \times k$ submatrix	Moderate
Circular Monotonic Deque	Duplicate array for circular windows	Circular sliding window	Moderate

16. Related Algorithms / Data Structures

Structure	Relationship	How to Choose
Monotonic Stack	Same principle, different structure	Use stack for NGE/PGE; deque for sliding window
Segment Tree	Range queries	Use deque for $O(n)$ sliding window; segment tree for arbitrary ranges
Priority Queue	Get max/min but $O(n \log k)$	Use deque for $O(n)$ when sliding window is fixed
Two Pointers	Sliding window without monotonicity	Use two pointers for sum-based window problems

17. Practice Problems

Easy

- **Next Greater Element I** — LeetCode (496) — Monotonic stack. Easy.
- **Baseball Game** — LeetCode (682) — Simple stack simulation. Easy.

Medium

- **Sliding Window Maximum** — LeetCode (239) — Monotonic deque. Medium.
- **Asteroid Collision** — LeetCode (735) — Stack simulation. Medium.

Hard

- **Largest Rectangle in Histogram** — LeetCode (84) — Monotonic stack. Hard.

- **Constrained Subsequence Sum** — LeetCode (1425) — DP + monotonic deque. Hard.

18. Interview Explanation

"A monotonic deque maintains elements in sorted order — either increasing or decreasing. When a new element arrives, we remove elements from the back that violate the monotonic property. This ensures that the front of the deque is always the answer (max or min) for the current window. Each element is added and removed at most once, giving $O(n)$ time. This pattern is used for sliding window max/min, next greater element, and related problems."

19. Revision Notes

- **Key idea:** Maintain sorted order in deque by removing dominated elements from back.
- **Decreasing deque:** Front is max. Pop from back when `arr[back] <= new`.
- **Increasing deque:** Front is min. Pop from back when `arr[back] >= new`.
- **Sliding window:** Also remove from front when out of range (`index <= i-k`).
- **Time:** $O(n)$, Space: $O(k)$ or $O(n)$.
- **Common trap:** Confusing increasing vs decreasing for min vs max.

20. Final Cheat Sheet

MONOTONIC DEQUE

```
When to use: Sliding window min/max, NGE, range queries
Invariant:   Elements in deque are sorted
Decreasing:  back cleanup when arr[back] <= new (for max)
Increasing:  back cleanup when arr[back] >= new (for min)
Complexity:  O(n) time, O(k) space
Key code:    while(!dq.empty() && arr[dq.back()] <= new)
              dq.pop_back();
              dq.push_back(i);
              if(dq.front() <= i-k) dq.pop_front();
Trap:        Store indices, not values
```

7. First Negative in Every Window

1. Overview

Given an array of integers and a window size `k`, find the **first negative number** in every contiguous subarray of size `k`. If a window has no negative number, output 0 (or some sentinel value).

This is a classic sliding window problem that tests queue-based thinking.

2. Intuition

Imagine sliding a window of size **k** over an array. At each position, you need to find the first negative number visible through the window.

The naive approach: for each window, scan from left to right until you find a negative number → $O(n \cdot k)$.

The queue approach: maintain a queue of **indices of negative numbers** that are currently in the window. The front of the queue is always the first negative in the current window. When the window slides:

- If the front index leaves the window, remove it.
- If a new negative number enters the window, add its index to the queue.

The key insight: **the first negative is always the earliest negative that is still in the window**. A queue perfectly captures the "first" property.

3. When to Use It

- When the problem asks for **"first negative in every window of size k"**.
- When you need to track the **first occurrence of a specific property** in a sliding window.
- When the property is **non-overlapping** (once a negative appears, it remains the "first negative" until it leaves the window).
- When you see **"first" + "sliding window"** in the problem.

4. When Not to Use It

- When you need **all negatives** in the window (not just first) — use a different approach.
- When you need the **last negative** in the window — use a similar approach but process from the right.
- When the window size is very small — $O(n \cdot k)$ is fine.
- When you need the **count** of negatives in each window — use prefix sum.
- When the array has no negatives — trivial, all answers are 0.

5. Core Concepts

Concept	Explanation	Why It Matters
Queue of Negative Indices	Stores indices of negative numbers in the current window	Front is always the first negative
Window Sliding	Remove front if out of window, add new negative if present	Maintains the invariant
Sentinel Value (0)	Default output when no negative exists in window	Required by the problem
FIFO Order	First negative in window = first negative that entered the window	Queue's FIFO property is perfect

6. Step-by-Step Algorithm

1. Initialize an empty queue **q** (will store indices of negative numbers).
2. Initialize an empty result array.
3. **First window** ($i = 0$ to $k-1$):
 - If `arr[i] < 0`, push **i** to **q**.

4. After processing first window:

- If `q` is not empty, result = `arr[q.front()]`, else result = 0.

5. **Remaining windows** ($i = k$ to $n-1$):

- **Remove out-of-window:** If `q` is not empty and `q.front() <= i - k`, pop front.
- **Add new element:** If `arr[i] < 0`, push `i` to `q`.
- **Record result:** If `q` is not empty, result = `arr[q.front()]`, else result = 0.

6. Return result.

7. Dry Run

Array: [12, -1, -7, 8, -15, 30, 16, 28], **k = 3**

Window (indices)	Window Elements	Queue of Negative Indices	First Negative
[0..2]	[12, -1, -7]	[1, 2]	-1
[1..3]	[-1, -7, 8]	[1, 2]	-1
[2..4]	[-7, 8, -15]	[2, 4]	-7
[3..5]	[8, -15, 30]	[4]	-15
[4..6]	[-15, 30, 16]	[4]	-15
[5..7]	[30, 16, 28]	[]	0

Step-by-step:

- **i=0 (12):** Not negative. `dq=[]`.
- **i=1 (-1):** Negative. `dq=[1]`.
- **i=2 (-7):** Negative. `dq=[1,2]`. First window: front = 1, `arr[1] = -1`. Result: [-1].
- **i=3 (8):** Not negative. Remove front if out of window: `dq[0]=1`, $i-k = 3-3=0$, $1 > 0$, keep. `dq=[1,2]`. Result: `arr[1] = -1`.
- **i=4 (-15):** Negative. Remove front if out: `dq[0]=1`, $i-k=4-3=1$, $1 \leq 1$, pop front. `dq=[2]`. Add 4. `dq=[2,4]`. Result: `arr[2] = -7`.
- **i=5 (30):** Not negative. Remove front if out: `dq[0]=2`, $i-k=5-3=2$, $2 \leq 2$, pop front. `dq=[4]`. Result: `arr[4] = -15`.
- **i=6 (16):** Not negative. Remove front if out: `dq[0]=4`, $i-k=6-3=3$, $4 > 3$, keep. `dq=[4]`. Result: `arr[4] = -15`.
- **i=7 (28):** Not negative. Remove front if out: `dq[0]=4`, $i-k=7-3=4$, $4 \leq 4$, pop front. `dq=[]`. Result: 0 (no negative).

Result: [-1, -1, -7, -15, -15, 0]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// First Negative in Every Window of Size K
vector<long long> firstNegativeInWindow(vector<long long>& arr, int k) {
```

```

int n = arr.size();
vector<long long> result;
queue<int> q; // stores indices of negative numbers

// Process first window
for (int i = 0; i < k; i++) {
    if (arr[i] < 0) {
        q.push(i);
    }
}

// Record result for first window
result.push_back(q.empty() ? 0 : arr[q.front()]);

// Process remaining windows
for (int i = k; i < n; i++) {
    // Remove out-of-window negative
    if (!q.empty() && q.front() <= i - k) {
        q.pop();
    }

    // Add new negative if present
    if (arr[i] < 0) {
        q.push(i);
    }

    // Record result for current window
    result.push_back(q.empty() ? 0 : arr[q.front()]);
}

return result;
}

// --- Example Usage ---
int main() {
    vector<long long> arr = {12, -1, -7, 8, -15, 30, 16, 28};
    int k = 3;

    vector<long long> result = firstNegativeInWindow(arr, k);

    cout << "First Negative in Each Window: ";
    for (long long x : result) {
        cout << x << " ";
    }
    // Output: -1 -1 -7 -15 -15 0
    cout << "\n";

    return 0;
}

```

9. Python Implementation

```
from collections import deque

def first_negative_in_window(arr, k):
    """First negative in every window of size k."""
    n = len(arr)
    result = []
    q = deque() # stores indices of negative numbers

    # Process first window
    for i in range(k):
        if arr[i] < 0:
            q.append(i)

    # Record result for first window
    result.append(arr[q[0]] if q else 0)

    # Process remaining windows
    for i in range(k, n):
        # Remove out-of-window negative
        if q and q[0] <= i - k:
            q.popleft()

        # Add new negative if present
        if arr[i] < 0:
            q.append(i)

        # Record result for current window
        result.append(arr[q[0]] if q else 0)

    return result

# Example usage
if __name__ == "__main__":
    arr = [12, -1, -7, 8, -15, 30, 16, 28]
    k = 3
    result = first_negative_in_window(arr, k)
    print("First Negative in Each Window:", result)
    # Output: [-1, -1, -7, -15, -15, 0]
```

10. Code Explanation

- **Queue of negative indices:** We maintain a queue containing only the indices of negative numbers that are currently in the window. The front of the queue is always the first negative number in the window.
- **First window (lines 7-9):** We scan the first k elements and push indices of all negative numbers into the queue.
- **Recording result:** After processing a window, if the queue is empty, no negative exists → output 0. Otherwise, the front of the queue gives the first negative.

- **Forward cleanup (lines 17-19):** When the window slides, we check if the front of the queue has left the window. If `q.front() <= i - k`, we pop it.
- **Adding new elements (lines 22-24):** If the new element (`arr[i]`) is negative, we push its index to the queue.
- **Why a queue works:** Negative numbers are discovered in order as the window slides. The first negative in the window is simply the earliest negative that hasn't left the window yet. The queue's FIFO property perfectly captures this.

11. Complexity Analysis

Aspect	Complexity
Time	$O(n)$ — each element is processed once
Space	$O(k)$ — queue holds at most k elements
Best case	$O(n)$ — no negatives, queue is always empty
Worst case	$O(n)$ — all elements are negative, queue has k elements

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
First Negative in Window	"First negative in every window of size k "	Queue of negative indices	GFG: First Negative in Every Window
First Positive in Window	"First positive in every window"	Same approach, opposite condition	Variation
First Element with Property P	"First element satisfying condition in window"	Queue of indices with property P	Generalization
Count of Negatives in Window	"Count of negatives in each window"	Prefix sum or sliding counter	Variation

13. Common Mistakes

- **Using 0 as sentinel when 0 is a valid value** — the problem may expect a different sentinel.
- **Not handling the case where no negative exists** — must output 0 (or sentinel).
- **Forgetting to remove out-of-window negatives** — stale indices give wrong results.
- **Using a deque (unnecessary)** — a simple queue is sufficient since we only remove from front.
- **Confusing "first negative" with "minimum"** — the first negative is the leftmost negative, not the smallest value.
- **Not using long long** — if the array can have large values, use `long long`.

14. Edge Cases

- Empty array: return empty.
- $k = 1$: each element is its own window. If negative, return that element; else 0.
- $k = n$: single window. Return first negative or 0.

- No negatives in array: all results are 0.
- All negatives: queue always has k elements, front is always the leftmost negative in the window.
- Single element, negative: return that element.
- Single element, positive: return 0.
- Large values: use `long long` to avoid overflow.

15. Variations

Variation	What Changes	When Used	Importance
First Positive in Window	Check for <code>arr[i] > 0</code> instead of <code>< 0</code>	Mirror problem	Low
Last Negative in Window	Process from right, or use deque with back	Alternative problem	Low
Count of Negatives in Window	Increment/decrement counter instead of queue	Simpler, just count	Moderate
First Negative in All Subarrays	Extend to all subarrays, not just size k	More complex	Low

16. Related Algorithms / Data Structures

Structure	Relationship	How to Choose
Queue (basic)	Simpler than deque, used for first-negative	Use queue for "first" problems
Deque	Needed when you need to remove from both ends	Use deque for max/min problems
Sliding Window (general)	Same sliding window technique	Use this for first-negative; two-pointer for sums
Prefix Sum	Can count negatives in range	Use prefix sum for count queries; queue for first occurrence

17. Practice Problems

Easy

- **First Negative in Every Window of Size K** — GFG — The classic problem. Easy.
- **Maximum of all subarrays of size K** — GFG — Sliding window max. Medium.

Medium

- **First Missing Positive** — LeetCode (41) — Not sliding window, but related to finding first. Hard.
- **Count Negative Numbers in a Sorted Matrix** — LeetCode (1351) — Different pattern. Easy.

Hard

- **First Missing Positive in Sliding Window** — Variation of the problem. Hard.

18. Interview Explanation

"The first negative in every window problem is solved by maintaining a queue of indices of negative numbers in the current window. The front of the queue is always the first negative. When the window slides, I remove the front if it's out of bounds, and add the new element if it's negative. The answer for each window is the value at the front of the queue, or 0 if the queue is empty. This runs in $O(n)$ time and $O(k)$ space."

19. Revision Notes

- **Key idea:** Queue of indices of negative numbers in the current window.
- **Front:** first negative in the window.
- **Empty queue:** no negative $\rightarrow$ output 0.
- **Updates:** pop front if out of window; push new index if negative.
- **Time:** $O(n)$, Space: $O(k)$.
- **Common trap:** Sentinel value (0) may differ by problem.

20. Final Cheat Sheet

FIRST NEGATIVE IN EVERY WINDOW

When to use: "First negative in every window of size k"

Data structure: Queue (indices of negative numbers)

Complexity: $O(n)$ time, $O(k)$ space

Key code:

```
if(arr[i] < 0) q.push(i);
if(!q.empty() && q.front() <= i-k) q.pop();
result = q.empty() ? 0 : arr[q.front()];
```

Edge cases: No negatives (all 0), all negatives (k elems)

Trap: Sentinel value – 0 may not always be correct

8. Rotten Oranges

1. Overview

Rotten Oranges (also called **Rotting Oranges** or **Oranges Rotting**) is a classic BFS problem on a grid. You are given a grid where each cell contains:

- 0 = empty cell
- 1 = fresh orange
- 2 = rotten orange

Every minute, any fresh orange that is **4-directionally adjacent** to a rotten orange becomes rotten. Find the minimum number of minutes until all oranges are rotten, or return -1 if impossible.

This is a **multi-source BFS** problem — we start BFS from all initially rotten oranges simultaneously.

2. Intuition

Imagine a box of oranges where some are already rotten. The rot spreads to adjacent oranges every minute, like a disease spreading in concentric waves.

This is exactly BFS where:

- All initially rotten oranges are at level 0.
- Oranges adjacent to them are at level 1 (they rot in minute 1).
- Oranges adjacent to those are at level 2, and so on.

The key insight: **the time when an orange rots is its shortest distance from any initially rotten orange**. This is a multi-source shortest path problem on an unweighted graph, which is exactly what BFS solves.

3. When to Use It

- When you have a **grid** with **sources** and the condition spreads to neighbors.
- When the spread is **uniform** (4-directional, same time per step).
- Problems asking for **"minimum time/distance for something to spread"**.
- Problems with **"multiple starting points"** that spread simultaneously.
- **Flood fill** variations where the fill spreads from multiple sources.
- Problems asking for **"distance to the nearest source"** for each cell.

4. When Not to Use It

- When the spread is **not uniform** (different weights per step) — use Dijkstra.
- When the spread is **probabilistic** — not a deterministic algorithm.
- When you need to find **if a path exists** between two specific cells — use DFS.
- When the grid is **very large** and you only need to know if all can be reached — BFS still works but may be memory-heavy.
- When the spread is **not 4-directional** (e.g., diagonal or arbitrary) — adjust the neighbor logic.

5. Core Concepts

Concept	Explanation	Why It Matters
Multi-Source BFS	BFS starting from multiple initial nodes simultaneously	All sources spread at the same rate
Level/Time Tracking	Distance from nearest source = time when orange rots	The answer is the maximum distance
4-Directional Adjacency	Up, down, left, right	Standard grid BFS movement
Fresh Orange Count	Track how many fresh oranges remain	If > 0 after BFS, return -1
Visited / Distance Grid	Tracks when each cell was visited	Prevents re-processing, tracks time

6. Step-by-Step Algorithm

1. Initialize a queue.
2. Count fresh oranges. Push all initially rotten oranges into the queue with time = 0.
3. Initialize a 2D distance array (or use the grid itself) to track time.
4. While queue is not empty:
 - Pop (x, y, time).
 - For each of the 4 neighbors:
 - If neighbor is a fresh orange (grid == 1):
 - Mark it as rotten (grid = 2).
 - Decrement fresh count.
 - Push neighbor with time + 1 into queue.
5. After BFS, if fresh count > 0, return -1.
6. Otherwise, return the maximum time recorded.

7. Dry Run

Grid:

```
2 1 1
1 1 0
0 1 1
```

Time	Queue (x, y, time)	Rotten Oranges	Fresh Left
0	(0,0,0)	(0,0)	6
0	(0,0,0) → (0,1,1), (1,0,1)	(0,0), (0,1), (1,0)	4
1	(0,1,1), (1,0,1) → (0,2,2), (1,1,2), (1,0 already)	(0,2), (1,1)	2
2	(1,0,1), (0,2,2), (1,1,2) → (2,0,3), (2,1,3)	(2,0), (2,1)	0

Wait — let me trace more carefully:

Initial: rotten at (0,0). Fresh at (0,1), (0,2), (1,0), (1,1), (1,2), (2,1), (2,2). Empty at (2,0).

BFS steps:

- t=0: Queue = [(0,0)]. Process (0,0). Neighbors: (0,1) fresh → rot, push (0,1). (1,0) fresh → rot, push (1,0). Fresh = 5.
- t=1: Queue = [(0,1), (1,0)]. Process (0,1). Neighbors: (0,2) fresh → rot, push (0,2). (1,1) fresh → rot, push (1,1). Fresh = 3. Process (1,0). Neighbors: (2,0) empty, skip. (1,1) already rotten. Fresh = 3.
- t=2: Queue = [(0,2), (1,1)]. Process (0,2). Neighbors: (1,2) fresh → rot, push (1,2). Fresh = 2. Process (1,1). Neighbors: (1,2) already rotten. (2,1) fresh → rot, push (2,1). (0,1) already rotten. Fresh = 1.
- t=3: Queue = [(1,2), (2,1)]. Process (1,2). Neighbors: (2,2) fresh → rot, push (2,2). Fresh = 0. Process (2,1). Neighbors: (2,2) already rotten.

Answer: 3 minutes (max time = 3).

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int orangesRotting(vector<vector<int>>& grid) {
    int n = grid.size(), m = grid[0].size();
    queue<pair<int,int>> q;
    int fresh = 0;
    int minutes = 0;

    // Direction arrays for 4-neighbor movement
    int dx[] = {-1, 1, 0, 0};
    int dy[] = {0, 0, -1, 1};

    // Initialize: push all rotten oranges, count fresh
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (grid[i][j] == 2) {
                q.push({i, j});
            } else if (grid[i][j] == 1) {
                fresh++;
            }
        }
    }

    // If no fresh oranges, return 0
    if (fresh == 0) return 0;

    // BFS
    while (!q.empty()) {
        int size = q.size();
        bool rotted = false;

        for (int s = 0; s < size; s++) {
            auto [x, y] = q.front();
            q.pop();

            for (int d = 0; d < 4; d++) {
                int nx = x + dx[d];
                int ny = y + dy[d];

                if (nx >= 0 && nx < n && ny >= 0 && ny < m && grid[nx][ny] == 1) {
                    grid[nx][ny] = 2;
                    fresh--;
                    q.push({nx, ny});
                    rotted = true;
                }
            }
        }

        if (rotted) minutes++;
    }
}
```

```

    return fresh == 0 ? minutes : -1;
}

// --- Example Usage ---
int main() {
    vector<vector<int>> grid = {
        {2, 1, 1},
        {1, 1, 0},
        {0, 1, 1}
    };

    int result = orangesRotting(grid);
    cout << "Minutes to rot all: " << result << "\n";
    // Output: 3

    return 0;
}

```

9. Python Implementation

```

from collections import deque

def oranges_rotting(grid):
    """Return minutes until all oranges rot, or -1 if impossible."""
    n, m = len(grid), len(grid[0])
    q = deque()
    fresh = 0
    minutes = 0

    dx = [-1, 1, 0, 0]
    dy = [0, 0, -1, 1]

    # Initialize: push all rotten oranges, count fresh
    for i in range(n):
        for j in range(m):
            if grid[i][j] == 2:
                q.append((i, j))
            elif grid[i][j] == 1:
                fresh += 1

    # If no fresh oranges, return 0
    if fresh == 0:
        return 0

    # BFS
    while q:
        size = len(q)
        rotted = False

        for _ in range(size):

```

```

        x, y = q.popleft()

        for d in range(4):
            nx, ny = x + dx[d], y + dy[d]

            if 0 <= nx < n and 0 <= ny < m and grid[nx][ny] == 1:
                grid[nx][ny] = 2
                fresh -= 1
                q.append((nx, ny))
                rotted = True

        if rotted:
            minutes += 1

    return minutes if fresh == 0 else -1

# Example usage
if __name__ == "__main__":
    grid = [
        [2, 1, 1],
        [1, 1, 0],
        [0, 1, 1]
    ]
    result = oranges_rotting(grid)
    print("Minutes to rot all:", result) # 3

```

10. Code Explanation

- **Multi-source BFS:** We push all initially rotten oranges into the queue. This ensures they all rot simultaneously.
- **Level-by-level processing:** We process the queue in levels (using `size = q.size()`) to track minutes. Each level corresponds to one minute.
- **Fresh orange count:** We track the number of fresh oranges. If it reaches 0 before BFS completes, we know the answer. If it's > 0 after BFS, some oranges are unreachable $\rightarrow$ return -1.
- **In-place modification:** We modify the grid in-place (set `grid[nx][ny] = 2`) to mark oranges as rotten. This serves as the visited array.
- **Direction arrays:** `dx` and `dy` encode the 4-directional movement. This is cleaner than writing 4 separate if statements.
- **rotted flag:** We only increment `minutes` if at least one orange rotted in this level. This prevents counting empty levels.

11. Complexity Analysis

Aspect	Complexity
Time	$O(N \times M)$ — each cell visited at most once
Space	$O(N \times M)$ — queue in worst case
Queue space	$O(N \times M)$ — all cells could be rotten

Aspect	Complexity
Fresh count	$O(1)$ — single integer

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Multi-Source BFS	Multiple starting points, same spread rate	Push all sources initially	01 Matrix, Rotting Oranges
Level-by-Level BFS	Need to track time/distance	Process queue in levels	Rotting Oranges, Word Ladder
Grid BFS with State	Grid with different cell types	BFS with direction arrays	Shortest Path in Binary Matrix
Distance to Nearest Source	"Distance to nearest 0" or "nearest 1"	Multi-source BFS from all sources	01 Matrix

13. Common Mistakes

- **Not counting fresh oranges** — without this, you can't detect impossible cases.
- **Not using multi-source BFS** — running BFS from each rotten orange separately gives wrong answer (they rot simultaneously, not sequentially).
- **Modifying grid while iterating** — can cause issues. Our BFS modifies only neighbors, not current cells.
- **Not checking bounds** in grid access.
- **Using DFS instead of BFS** — DFS doesn't give shortest time.
- **Confusing minutes with levels** — each level = 1 minute, but only if at least one orange rotted in that level.
- **Returning minutes-1** — off-by-one error in time calculation.

14. Edge Cases

- No fresh oranges: return 0 immediately.
- No rotten oranges: return -1 (impossible).
- Single cell, fresh: return -1 (no rotten to spread).
- Single cell, rotten: return 0 (no fresh to rot).
- All rotten: return 0.
- Fresh oranges isolated by empty cells: return -1.
- Empty grid: handle gracefully (depends on constraints).
- Large grid: ensure $O(N \times M)$ time and space.

15. Variations

Variation	What Changes	When Used	Importance
01 Matrix	Find distance to nearest 0 for each cell	Multi-source BFS from all 0s	Very High

Variation	What Changes	When Used	Importance
As Far from Land as Possible	Find max distance from any land	Multi-source BFS from all lands	Moderate
Shortest Path to Get All Keys	BFS with state (bitmask)	Complex grid puzzle	High
Pacific Atlantic Water Flow	BFS from both oceans	Two BFS runs	High

16. Related Algorithms / Data Structures

Structure	Relationship	How to Choose
BFS	Core algorithm	Use for unweighted shortest path
Multi-Source BFS	Extension of BFS	Use when multiple sources exist
Dijkstra	Weighted shortest path	Use when spread time is not uniform
DFS	Path existence, connectivity	Use for different problems
Union-Find (DSU)	Connectivity queries	Use for dynamic connectivity

17. Practice Problems

Easy

- **Flood Fill** — LeetCode (733) — Simple BFS/DFS on grid. Easy.
- **Island Perimeter** — LeetCode (463) — Grid traversal. Easy.

Medium

- **Rotting Oranges** — LeetCode (994) — The classic problem. Medium.
- **01 Matrix** — LeetCode (542) — Multi-source BFS. Medium.

Hard

- **Shortest Path to Get All Keys** — LeetCode (864) — BFS with bitmask state. Hard.
- **Minimum Number of Flips to Convert Binary Matrix** — LeetCode (1284) — BFS with state. Hard.

18. Interview Explanation

"Rotting Oranges is a multi-source BFS problem. All rotten oranges spread simultaneously, so we push all of them into the queue initially. I process the queue level by level, where each level corresponds to one minute. For each rotten orange, I check its 4 neighbors — if any is fresh, it becomes rotten and gets pushed into the queue for the next level. I track the number of fresh oranges remaining. If after BFS any fresh oranges remain, it's impossible and I return -1. Otherwise, I return the number of levels processed. Time complexity is $O(N \times M)$."

19. Revision Notes

- **Key idea:** Multi-source BFS, level-by-level processing.
- **Data structure:** Queue of (x, y).
- **Fresh count:** Track to detect impossible cases.
- **Level processing:** Each level = 1 minute.
- **Return:** max minutes if all rotted, else -1.
- **Time:** $O(N \times M)$, Space: $O(N \times M)$.
- **Common trap:** Forgetting multi-source (all initial rotten oranges go in first).

20. Final Cheat Sheet

ROTTEN ORANGES / MULTI-SOURCE BFS

When to use: Grid with multiple sources, uniform spread

Data structure: Queue (multi-source BFS)

Complexity: $O(N \times M)$ time, $O(N \times M)$ space

Key code:

```
q.push(all initially rotten);
while(!q.empty()) { size = q.size();
  for(s in 0..size) { pop, process neighbors }
  minutes++; }
```

Return: `fresh==0 ? minutes : -1`

Edge cases: No fresh (0), no rotten (-1), isolated fresh

Trap: Push ALL sources initially, not just one

9. Task Scheduler

1. Overview

You are given a list of tasks (each task is a letter A–Z) and a cooling interval `n`. After executing a task, you must wait `n` units before executing the **same** task again. You can execute any other task during the cooling period. Find the **minimum number of time units** needed to execute all tasks.

This is a scheduling problem that can be solved using a **max-heap** (priority queue) and a **queue** for the cooling period.

2. Intuition

Imagine you have different types of tasks. Each task takes 1 unit of time. After doing a task of type A, you cannot do A again for `n` units. But you can do B, C, etc. during that time.

The greedy approach: **always execute the task with the highest remaining frequency first**. This minimizes idle time because we're keeping the CPU busy with the most frequent tasks.

- Use a max-heap to always get the most frequent remaining task.
- Use a queue to track tasks that are currently cooling down.
- At each time unit:

- If the heap is not empty, pick the most frequent task and execute it.
- Decrement its frequency. If it still has remaining tasks, push it into the cooling queue with time `current + n`.
- If the front of the cooling queue's time has arrived, push it back into the heap.

The key insight: **idle time occurs when the most frequent task is on cooldown and no other tasks are available.**

3. When to Use It

- When you have **tasks with different frequencies** and a **cooldown period** between same tasks.
- Problems asking for "**minimum time to complete all tasks**" with constraints.
- **CPU scheduling** problems with cooldown.
- Problems where you need to **rearrange tasks** to minimize time.
- When you see "**cooling interval**" or "**cooldown period**" in the problem.
- Problems like "**rearrange string so that no two same characters are k apart**".

4. When Not to Use It

- When tasks have **different execution times** (not just 1 unit each) — use a different scheduling algorithm.
- When tasks have **dependencies** (must execute A before B) — use topological sort.
- When there is **no cooldown** ($n = 0$) — answer is simply the number of tasks.
- When you need to return the **actual schedule** (not just the time) — the same algorithm can be extended.
- When the number of task types is very large — the heap size is still $O(26)$ at most, which is fine.

5. Core Concepts

Concept	Explanation	Why It Matters
Max-Heap (Priority Queue)	Stores frequencies of tasks, highest frequency first	Greedy — always pick the most frequent task
Cooling Queue	Queue of (frequency, ready_time) for tasks on cooldown	Tracks when a task can be executed again
Idle Time	When no task is available to execute	Must be added to the schedule
Frequency	Count of remaining executions for each task	Determines priority
Cycle	Cooldown period n	The constraint that makes scheduling interesting

6. Step-by-Step Algorithm

1. Count frequencies of all tasks.
2. Push all frequencies into a max-heap.
3. Initialize `time = 0`.

4. Initialize a queue `cooling` (stores `(freq, ready_time)`).
5. While heap is not empty or cooling queue is not empty:
 - If cooling queue is not empty and `cooling.front().ready_time == time`:
 - Push `cooling.front().freq` back into heap.
 - Pop from cooling queue.
 - If heap is not empty:
 - Pop the max frequency `freq`.
 - If `freq > 1`: push `(freq - 1, time + n + 1)` into cooling queue.
 - Else (heap is empty):
 - Idle time — skip to next available task.
 - `time++`.
6. Return `time`.

7. Dry Run

Tasks: ['A', 'A', 'A', 'B', 'B', 'C'], **n = 2**

Frequencies: A=3, B=2, C=1. Max-heap: [3, 2, 1].

Time	Heap	Cooling Queue	Task Executed	Notes
0	[3,2,1]	[]	A (freq=2)	A pushed to cooling: (2, 3)
1	[2,1]	[(2, 3)]	B (freq=1)	B pushed to cooling: (1, 4)
2	[1]	[(2, 3), (1, 4)]	C (freq=0)	Done with C
3	[1]	[(1, 4)]	A (freq=1)	Cooling: (2, 3) → ready, push 2 to heap. A pushed to cooling: (1, 6)
4	[2]	[(1, 6)]	B (freq=0)	Cooling: (1, 4) → ready, push 1 to heap. Done with B.
5	[1]	[(1, 6)]	A (freq=0)	Done with A.
6	[]	[]	—	All done.

Wait, let me re-trace more carefully:

Initial: heap = [3, 2, 1], cooling = [], time = 0.

- **t=0:** Pop 3 (A). freq-1 = 2. Push (2, 0+2+1=3) to cooling. heap = [2, 1]. time = 1.
- **t=1:** Pop 2 (B). freq-1 = 1. Push (1, 1+2+1=4) to cooling. heap = [1]. time = 2.
- **t=2:** Pop 1 (C). freq-1 = 0, done. heap = []. time = 3.
- **t=3:** cooling front: (2, 3) is ready. Push 2 to heap. Pop 2 (A). freq-1 = 1. Push (1, 3+2+1=6). heap = [1]. time = 4.
- **t=4:** cooling front: (1, 4) is ready. Push 1 to heap. heap = [1, 1]. cooling = [(1, 6)]. Pop 1 (B). freq-1 = 0, done. heap = [1]. time = 5.
- **t=5:** No cooling ready. Pop 1 (A). freq-1 = 0, done. heap = []. time = 6.
- **t=6:** cooling = [(1, 6)]. Front (1, 6) is ready. Push 1 to heap. Pop 1 (B). freq-1 = 0, done. time = 7.

Hmm, that gives 7. Let me reconsider.

Actually, the standard solution for this problem is simpler. Let me use the formula-based approach:

Optimal approach: The minimum time is determined by the most frequent task.

Let `maxFreq` = frequency of the most frequent task. Let `maxCount` = number of tasks with frequency = `maxFreq`.

The formula: `time = max(tasks.size(), (maxFreq - 1) * (n + 1) + maxCount)`

For A=3, B=2, C=1, n=2:

- `maxFreq` = 3 (A), `maxCount` = 1 (only A has frequency 3)
- $(3-1) * (2+1) + 1 = 2 * 3 + 1 = 7$
- `tasks.size()` = 6
- $\max(6, 7) = 7$

The schedule: A, B, C, A, B, idle, A → that's 7 units.

But we can do better with a different arrangement: A, B, idle, A, B, idle, A → still 7.

Actually the optimal schedule for A=3, B=2, C=1, n=2: A, B, C, A, B, idle, A = 7 units.

Yes, 7 is correct.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Task Scheduler using Max-Heap + Queue
int leastInterval(vector<char>& tasks, int n) {
    // Count frequencies
    vector<int> freq(26, 0);
    for (char c : tasks) {
        freq[c - 'A']++;
    }

    // Max-heap of frequencies
    priority_queue<int> pq;
    for (int f : freq) {
        if (f > 0) pq.push(f);
    }

    // Queue of (freq, ready_time)
    queue<pair<int, int>> cooling;
    int time = 0;

    while (!pq.empty() || !cooling.empty()) {
        // Check if any task has finished cooling
        if (!cooling.empty() && cooling.front().second == time) {
            pq.push(cooling.front().first);
            cooling.pop();
        }
    }
}
```

```

        if (!pq.empty()) {
            int f = pq.top();
            pq.pop();
            if (f > 1) {
                cooling.push({f - 1, time + n + 1});
            }
        }
        // else: idle - time passes

        time++;
    }

    return time;
}

// Alternative: Formula-based approach (O(n) time, O(1) space)
int leastIntervalFormula(vector<char>& tasks, int n) {
    vector<int> freq(26, 0);
    int maxFreq = 0;

    for (char c : tasks) {
        freq[c - 'A']++;
        maxFreq = max(maxFreq, freq[c - 'A']);
    }

    // Count how many tasks have max frequency
    int maxCount = 0;
    for (int f : freq) {
        if (f == maxFreq) maxCount++;
    }

    // Formula: (maxFreq-1) * (n+1) + maxCount
    int result = (maxFreq - 1) * (n + 1) + maxCount;

    // Result cannot be less than total number of tasks
    return max((int)tasks.size(), result);
}

// --- Example Usage ---
int main() {
    vector<char> tasks = {'A', 'A', 'A', 'B', 'B', 'C'};
    int n = 2;

    cout << "Least Interval (Heap): " << leastInterval(tasks, n) << "\n";
    cout << "Least Interval (Formula): " << leastIntervalFormula(tasks, n) <<
    "\n";
    // Output: 7

    return 0;
}

```

9. Python Implementation

```

from collections import deque
import heapq

def least_interval(tasks, n):
    """Return minimum time to execute all tasks with cooldown n."""
    freq = [0] * 26
    for c in tasks:
        freq[ord(c) - ord('A')] += 1

    # Max-heap (store negative values for Python's min-heap)
    pq = [-f for f in freq if f > 0]
    heapq.heapify(pq)

    cooling = deque() # (freq, ready_time)
    time = 0

    while pq or cooling:
        # Check if any task has finished cooling
        if cooling and cooling[0][1] == time:
            f, _ = cooling.popleft()
            heapq.heappush(pq, -f)

        if pq:
            f = -heapq.heappop(pq)
            if f > 1:
                cooling.append((f - 1, time + n + 1))

        time += 1

    return time

def least_interval_formula(tasks, n):
    """Formula-based approach: O(n) time, O(1) space."""
    freq = [0] * 26
    for c in tasks:
        freq[ord(c) - ord('A')] += 1

    max_freq = max(freq)
    max_count = freq.count(max_freq)

    result = (max_freq - 1) * (n + 1) + max_count
    return max(len(tasks), result)

# Example usage
if __name__ == "__main__":
    tasks = ['A', 'A', 'A', 'B', 'B', 'C']
    n = 2

```

```
print("Least Interval (Heap):", least_interval(tasks, n)) # 7
print("Least Interval (Formula):", least_interval_formula(tasks, n)) # 7
```

10. Code Explanation

Heap + Queue approach:

- **Frequency counting:** We count how many times each task appears.
- **Max-heap:** We push all frequencies into a max-heap. This ensures we always pick the most frequent task available.
- **Cooling queue:** Each entry is `(freq, ready_time)`. After executing a task, its remaining frequency (if > 0) goes into the cooling queue with a ready time of `current_time + n + 1`.
- **Main loop:** At each time step, we check if any task in the cooling queue is ready. If so, we push it back into the heap. Then we execute the most frequent task from the heap.
- **Idle time:** If the heap is empty and the cooling queue has tasks, we idle (time passes).

Formula approach (much simpler):

- The most frequent task determines the minimum time. If the most frequent task appears `maxFreq` times, there are `maxFreq - 1` gaps between its occurrences. Each gap must be at least `n` units long. So the total time is `(maxFreq - 1) * (n + 1) + maxCount`, where `maxCount` is the number of tasks with `maxFreq` frequency.
- This formula works because the gaps are filled with other tasks. If there aren't enough other tasks, idle time is added.
- The answer cannot be less than the total number of tasks.

11. Complexity Analysis

Approach	Time	Space
Heap + Queue	$O(N \times \log 26) = O(N)$	$O(26) = O(1)$
Formula	$O(N)$	$O(26) = O(1)$

Detailed:

- Counting frequencies: $O(N)$
- Heap operations: $O(\log 26) = O(1)$ per operation (since max 26 letters)
- Queue operations: $O(1)$ per operation
- Formula: $O(N + 26) = O(N)$

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Task Scheduling with Cooldown	"Cooldown period", "same task cannot repeat"	Max-heap + cooling queue	LeetCode 621

Pattern	How to Identify	Approach	Example Problems
Rearrange String with Distance	"No two same characters are k apart"	Greedy with priority queue	LeetCode 358
CPU Scheduling	"Process scheduling", "round-robin"	Queue-based simulation	Various
Maximum Frequency Scheduling	"Always pick most frequent"	Max-heap	Huffman Coding

13. Common Mistakes

- **Forgetting the formula** — the formula makes the problem $O(N)$ and is much simpler.
- **Not considering the total tasks count** — the formula result must be compared with `tasks.size()`.
- **Using n instead of $n+1$** in the formula — the cycle length is $n+1$ (task + n idle).
- **Not handling the case where heap is empty** — idle time must be counted.
- **Confusing cooling queue with priority queue** — tasks in cooling are not available for execution.
- **Incorrect ready_time calculation** — should be $\text{time} + n + 1$, not $\text{time} + n$.

14. Edge Cases

- $n = 0$: no cooldown $\rightarrow$ answer = number of tasks.
- All same task: e.g., A, A, A with $n=2 \rightarrow$ A, idle, idle, A, idle, idle, A = 7. Formula: $(3-1)*(2+1)+1 = 7$.
- All different tasks: no cooldown issues $\rightarrow$ answer = number of tasks.
- Single task: answer = 1.
- Empty tasks: answer = 0.
- Large n : may have many idle slots.
- $\text{maxCount} > 1$: multiple tasks with the same max frequency $\rightarrow$ they all appear at the end.

15. Variations

Variation	What Changes	When Used	Importance
Task Scheduler II	Tasks have different cooldowns for each task type	More complex scheduling	Moderate
Rearrange String k Distance Apart	Rearrange string, not just count time	Return the actual arrangement	High
Minimum Time to Complete All Tasks	Tasks have different execution times	More general scheduling	Moderate
Maximum Number of Tasks You Can Complete	Task selection with deadlines	Different optimization	Moderate

16. Related Algorithms / Data Structures

Structure	Relationship	How to Choose
Max-Heap (Priority Queue)	Core data structure	Always pick the most frequent task
Queue	For cooling period	Track when tasks become available again
Greedy	Algorithmic paradigm	Always pick the highest frequency task
Counting Sort	Alternative simpler approach	Use the formula for $O(N)$ time

17. Practice Problems

Easy

- **Minimum Time to Complete All Tasks** — Simple scheduling. Easy.
- **Number of Tasks You Can Complete** — Greedy scheduling. Easy.

Medium

- **Task Scheduler** — LeetCode (621) — The classic problem. Medium.
- **Rearrange String k Distance Apart** — LeetCode (358) — Similar pattern. Hard.

Hard

- **Minimum Number of Taps to Open to Water a Garden** — LeetCode (1326) — Greedy + scheduling. Hard.
- **Maximum Number of Events That Can Be Attended** — LeetCode (1353) — Greedy scheduling. Hard.

18. Interview Explanation

"Task Scheduler can be solved with a greedy approach using a max-heap and a cooling queue. The most frequent task determines the minimum time. I prefer the formula: $(\text{maxFreq} - 1) * (n + 1) + \text{maxCount}$, clamped to the total number of tasks. The intuition is that the most frequent task creates $\text{maxFreq} - 1$ gaps, each of length n , and the last occurrence of the most frequent tasks are at the end. If there are enough other tasks to fill the gaps, no idle time is needed. The heap approach is more general and handles edge cases but the formula is $O(N)$ and much simpler."

19. Revision Notes

- **Key idea:** Most frequent task determines minimum time.
- **Formula:** $(\text{maxFreq} - 1) * (n + 1) + \text{maxCount}$.
- **Clamp:** $\text{max}(\text{formula}, \text{total_tasks})$.
- **Heap approach:** Max-heap of frequencies + cooling queue.
- **Time:** $O(N)$ for formula, $O(N \log 26)$ for heap.
- **Space:** $O(1)$.
- **Common trap:** Using n instead of $n+1$ in the formula.

20. Final Cheat Sheet

TASK SCHEDULER

When to use: Task scheduling with cooldown period

Formula: $(\text{maxFreq}-1)*(n+1) + \text{maxCount}$

Clamp: $\max(\text{formula}, \text{total_tasks})$

Complexity: $O(N)$ time, $O(1)$ space (formula)

Key code: `result = (maxFreq-1)*(n+1) + maxCount;
return max(tasks.size(), result);`

Edge cases: $n=0$ (all tasks), all same (max idle), all diff

Trap: Use $n+1$, not n , in the cycle length

10. 0-1 BFS Using Deque

1. Overview

0-1 BFS is a variation of BFS used to find the shortest path in a graph where edge weights are either **0 or 1**. It uses a **deque** instead of a regular queue and runs in $O(V + E)$ time — faster than Dijkstra's $O((V+E) \log V)$ for this special case.

This is a powerful technique for CP and placements when dealing with grids or graphs with 0/1 cost edges.

2. Intuition

In regular BFS, all edges have equal weight, so we use a queue. In Dijkstra, edges have arbitrary weights, so we use a priority queue.

0-1 BFS is the middle ground. Since edges are only 0 or 1, we can use a **deque**:

- When we traverse a **0-weight edge**, the distance doesn't change, so we push the new node to the **front** of the deque (it should be processed before nodes with higher distance).
- When we traverse a **1-weight edge**, the distance increases by 1, so we push the new node to the **back** of the deque (it should be processed after nodes with the same distance).

This maintains the invariant that the deque always has nodes sorted by distance (non-decreasing from front to back). This is equivalent to Dijkstra but much faster.

3. When to Use It

- When the graph has **edge weights of only 0 or 1**.
- When you see a **grid with 0/1 cost** for moving between cells.
- Problems where **some moves are free (cost 0)** and **others cost 1**.
- When you need **shortest path** in a graph with small integer weights.
- In **CP problems** where the weight range is limited (0-1, 0-2, etc.) — can be extended to 0-k BFS.
- When Dijkstra is an option but $O((V+E) \log V)$ is too slow.

4. When Not to Use It

- When all edges have **equal weight** (use regular BFS — simpler).
- When edge weights are **arbitrary positive integers** (use Dijkstra).
- When edge weights can be **negative** (use Bellman-Ford or SPFA).
- When the graph is **unweighted** — regular BFS is simpler and also $O(V+E)$.
- When you need to find **all shortest paths** (not just distance) — 0-1 BFS still works but you need to track parents.

5. Core Concepts

Concept	Explanation	Why It Matters
Deque	Double-ended queue	Allows $O(1)$ push_front and push_back
0-weight edge → push_front	Distance doesn't change, process immediately	Maintains sorted order by distance
1-weight edge → push_back	Distance increases by 1, process later	Maintains sorted order by distance
Distance array	Stores shortest distance from source	Initialized to INF
Invariant	Deque is always sorted by distance (front = min, back = max)	Ensures first time we visit a node is via shortest path
Dial's Algorithm	Extension to k different weights	Uses buckets instead of deque

6. Step-by-Step Algorithm

1. Initialize a distance array `dist` with INF for all nodes. Set `dist[source] = 0`.
2. Initialize a deque. Push `source` to the front.
3. While the deque is not empty:
 - a. Pop the front node `u`.
 - b. For each neighbor `v` of `u` with edge weight `w` (0 or 1):
 - If `dist[u] + w < dist[v]`:
 - Update `dist[v] = dist[u] + w`.
 - If `w == 0`: push `v` to the **front** of the deque.
 - If `w == 1`: push `v` to the **back** of the deque.
4. Return `dist` array.

7. Dry Run

Graph: 5 nodes. Edges:

- $0 \rightarrow 1$ (weight 0)
- $0 \rightarrow 2$ (weight 1)
- $1 \rightarrow 2$ (weight 0)
- $1 \rightarrow 3$ (weight 1)
- $2 \rightarrow 3$ (weight 0)
- $2 \rightarrow 4$ (weight 1)

- $3 \rightarrow 4$ (weight 0)

Source = 0.

Step	Deque (front $\rightarrow$ back)	Node Popped	dist[]
Init	[0]	—	[0, ∞ , ∞ , ∞ , ∞]
1	[]	0	[0, ∞ , ∞ , ∞ , ∞]
2	[1]	—	[0, 0, ∞ , ∞ , ∞] (0 $\rightarrow$ 1, w=0 $\rightarrow$ push_front)
3	[2]	—	[0, 0, 1, ∞ , ∞] (0 $\rightarrow$ 2, w=1 $\rightarrow$ push_back)
4	[1, 2]	—	(from step 2, but we haven't processed 1 yet)

Wait, let me trace more carefully.

Initial: dist = [0, ∞ , ∞ , ∞ , ∞]. dq = [0].

- **Pop 0.** Neighbors:
 - 1 (w=0): dist[1] = $\infty > 0+0 = 0$. Update dist[1]=0. Push_front(1). dq = [1].
 - 2 (w=1): dist[2] = $\infty > 0+1 = 1$. Update dist[2]=1. Push_back(2). dq = [1, 2].
- **Pop 1 (front).** Neighbors:
 - 2 (w=0): dist[2] = 1 $> 0+0 = 0$. Update dist[2]=0. Push_front(2). dq = [2, 2]... wait, we have two 2s in the deque.

Hmm, actually the standard 0-1 BFS doesn't check for duplicates in the deque. This can lead to the same node being in the deque multiple times. But each time we process a node, we check if the distance we found is better. If it's not, we skip.

Actually, let me re-check. The standard implementation does:

```
if (dist[v] > dist[u] + w) {
    dist[v] = dist[u] + w;
    if (w == 0) dq.push_front(v);
    else dq.push_back(v);
}
```

This means node 2 could be pushed twice — once from 0 (dist=1) pushed to back, then later from 1 (dist=0) pushed to front. When we later pop the first occurrence of 2 (with dist=1), we check if dist[2] == 1, but it's now 0, so we skip processing. When we pop the second occurrence (with dist=0), we process normally.

Let me continue the trace properly:

Initial: dist = [0, ∞ , ∞ , ∞ , ∞]. dq = [0].

- **Pop 0.** Process neighbors:
 - 1 (w=0): dist[1]= $\infty > 0+0=0 \rightarrow$ dist[1]=0, push_front(1). dq = [1].

- 2 ($w=1$): $\text{dist}[2]=\infty > 0+1=1 \rightarrow \text{dist}[2]=1$, $\text{push_back}(2)$. $\text{dq} = [1, 2]$.
- **Pop 1 (front).** $\text{dist}[1] = 0$, matches. Process neighbors:
 - 2 ($w=0$): $\text{dist}[2]=1 > 0+0=0 \rightarrow \text{dist}[2]=0$, $\text{push_front}(2)$. $\text{dq} = [2, 2]$.
 - 3 ($w=1$): $\text{dist}[3]=\infty > 0+1=1 \rightarrow \text{dist}[3]=1$, $\text{push_back}(3)$. $\text{dq} = [2, 2, 3]$.
- **Pop 2 (front).** $\text{dist}[2] = 0$, matches. Process neighbors:
 - 3 ($w=0$): $\text{dist}[3]=1 > 0+0=0 \rightarrow \text{dist}[3]=0$, $\text{push_front}(3)$. $\text{dq} = [3, 2, 3]$.
 - 4 ($w=1$): $\text{dist}[4]=\infty > 0+1=1 \rightarrow \text{dist}[4]=1$, $\text{push_back}(4)$. $\text{dq} = [3, 2, 3, 4]$.
- **Pop 3 (front).** $\text{dist}[3] = 0$, matches. Process neighbors:
 - 4 ($w=0$): $\text{dist}[4]=1 > 0+0=0 \rightarrow \text{dist}[4]=0$, $\text{push_front}(4)$. $\text{dq} = [4, 2, 3, 4]$.
- **Pop 4 (front).** $\text{dist}[4] = 0$, matches. Process neighbors... (No more relevant updates)

The remaining pops (2, 3, 4 from the back) will have stale distances and will be skipped.

Final distances: $\text{dist} = [0, 0, 0, 0, 0]$ — all reachable with 0 cost from source 0 via 0-weight edges.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// 0-1 BFS on a graph
vector<int> zeroOneBFS(int n, vector<pair<int,int>> adj[], int source) {
    vector<int> dist(n, INT_MAX);
    deque<int> dq;

    dist[source] = 0;
    dq.push_front(source);

    while (!dq.empty()) {
        int u = dq.front();
        dq.pop_front();

        for (auto &[v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                if (w == 0) {
                    dq.push_front(v); // 0-weight → process immediately
                } else {
                    dq.push_back(v);  // 1-weight → process later
                }
            }
        }
    }

    return dist;
}
```

```

// 0-1 BFS on a grid (moving up/down/left/right with costs)
// Grid: 0 = free cell, 1 = cost 1 to enter
int zeroOneBFSGrid(vector<vector<int>>& grid, pair<int,int> source, pair<int,int>
target) {
    int n = grid.size(), m = grid[0].size();
    vector<vector<int>> dist(n, vector<int>(m, INT_MAX));
    deque<pair<int,int>> dq;

    int dx[] = {-1, 1, 0, 0};
    int dy[] = {0, 0, -1, 1};

    dist[source.first][source.second] = 0;
    dq.push_front(source);

    while (!dq.empty()) {
        auto [x, y] = dq.front();
        dq.pop_front();

        if (x == target.first && y == target.second) {
            return dist[x][y];
        }

        for (int d = 0; d < 4; d++) {
            int nx = x + dx[d];
            int ny = y + dy[d];

            if (nx >= 0 && nx < n && ny >= 0 && ny < m) {
                int w = grid[nx][ny]; // cost to enter this cell (0 or 1)
                if (dist[x][y] + w < dist[nx][ny]) {
                    dist[nx][ny] = dist[x][y] + w;
                    if (w == 0) {
                        dq.push_front({nx, ny});
                    } else {
                        dq.push_back({nx, ny});
                    }
                }
            }
        }
    }

    return -1; // unreachable
}

// --- Example Usage ---
int main() {
    // Graph with 0/1 weights
    int n = 5;
    vector<pair<int,int>> adj[n];

    // 0 → 1 (0), 0 → 2 (1)
    adj[0] = {{1, 0}, {2, 1}};
    // 1 → 2 (0), 1 → 3 (1)
    adj[1] = {{2, 0}, {3, 1}};

```

```

// 2 → 3 (0), 2 → 4 (1)
adj[2] = {{3, 0}, {4, 1}};
// 3 → 4 (0)
adj[3] = {{4, 0}};

vector<int> dist = zeroOneBFS(n, adj, 0);

for (int i = 0; i < n; i++) {
    cout << "Dist to " << i << ": " << dist[i] << "\n";
}

// Grid example
vector<vector<int>> grid = {
    {0, 0, 1},
    {1, 0, 0},
    {0, 1, 0}
};

int result = zeroOneBFSGrid(grid, {0, 0}, {2, 2});
cout << "Shortest path cost in grid: " << result << "\n";

return 0;
}

```

9. Python Implementation

```

from collections import deque

def zero_one_bfs(n, adj, source):
    """0-1 BFS on a graph. adj is list of lists of (neighbor, weight)."""
    dist = [float('inf')] * n
    dq = deque()

    dist[source] = 0
    dq.appendleft(source)

    while dq:
        u = dq.popleft()

        for v, w in adj[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                if w == 0:
                    dq.appendleft(v) # 0-weight → process immediately
                else:
                    dq.append(v)     # 1-weight → process later

    return dist

```

```

def zero_one_bfs_grid(grid, source, target):
    """0-1 BFS on a grid. 0 = free, 1 = cost 1 to enter."""
    n, m = len(grid), len(grid[0])
    dist = [[float('inf')] * m for _ in range(n)]
    dq = deque()

    sx, sy = source
    dist[sx][sy] = 0
    dq.appendleft((sx, sy))

    while dq:
        x, y = dq.popleft()

        if (x, y) == target:
            return dist[x][y]

        for d in range(4):
            nx, ny = x + dx[d], y + dy[d]

            if 0 <= nx < n and 0 <= ny < m:
                w = grid[nx][ny] # cost to enter this cell
                if dist[x][y] + w < dist[nx][ny]:
                    dist[nx][ny] = dist[x][y] + w
                    if w == 0:
                        dq.appendleft((nx, ny))
                    else:
                        dq.append((nx, ny))

    return -1 # unreachable

# Example usage
if __name__ == "__main__":
    # Graph
    n = 5
    adj = [
        [(1, 0), (2, 1)],
        [(2, 0), (3, 1)],
        [(3, 0), (4, 1)],
        [(4, 0)],
        []
    ]
    dist = zero_one_bfs(n, adj, 0)
    for i, d in enumerate(dist):
        print(f"Dist to {i}: {d}")

    # Grid
    grid = [
        [0, 0, 1],
        [1, 0, 0],
        [0, 1, 0]
    ]

```

```
]
result = zero_one_bfs_grid(grid, (0, 0), (2, 2))
print("Shortest path cost in grid:", result)
```

10. Code Explanation

- **Deque:** The core data structure. We push 0-weight edges to the front and 1-weight edges to the back. This maintains the invariant that the deque is sorted by distance (non-decreasing from front to back).
- **Distance array:** Initialized to INF. When we find a shorter path to a node, we update the distance and push the node to the deque.
- **Push to front (0-weight):** If the edge weight is 0, the distance to the neighbor is the same as the current node. This neighbor should be processed before nodes with higher distance, so we push it to the front.
- **Push to back (1-weight):** If the edge weight is 1, the distance increases by 1. This neighbor should be processed after all nodes with the current distance, so we push it to the back.
- **Stale entries:** A node may be pushed to the deque multiple times if we find a shorter path later. When we pop a stale entry, the distance check `dist[u] + w < dist[v]` will fail because `dist[v]` is already better, so we skip it.
- **Grid version:** The cost to enter a cell is the value of that cell (0 or 1). This is the edge weight.

11. Complexity Analysis

Aspect	Complexity
Time	$O(V + E)$ — each edge is processed at most once
Space	$O(V)$ — distance array + deque
Best case	$O(V + E)$ — same
Worst case	$O(V + E)$ — same

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
0-1 BFS on Graph	Edge weights are 0 or 1	Deque, push_front for 0, push_back for 1	Minimum Cost to Make at Least One Valid Path
0-1 BFS on Grid	Grid with 0/1 cost cells	Deque, cost = value of target cell	Minimum Cost to Reach Destination
Minimum Cost to Change Direction	Path with direction changes (cost 0 = same direction, cost 1 = turn)	0-1 BFS on directional graph	Minimum Cost to Make at Least One Valid Path
Dial's Algorithm (0-k BFS)	Edge weights are 0, 1, ..., k	Use array of buckets, not deque	Shortest path with small integer weights

13. Common Mistakes

- **Using regular queue instead of deque** — the algorithm won't work correctly.
- **Pushing 0-weight edges to the back** — should go to the front.
- **Pushing 1-weight edges to the front** — should go to the back.
- **Not checking for stale entries** — processing a node with outdated distance leads to incorrect results.
- **Confusing 0-1 BFS with Dijkstra** — 0-1 BFS is $O(V+E)$ and uses a deque; Dijkstra is $O((V+E) \log V)$ and uses a priority queue.
- **Forgetting to initialize dist array with INF** — uninitialized values cause wrong comparisons.
- **Not handling the case where 0-weight edges create cycles** — the distance check prevents infinite loops.

14. Edge Cases

- All edges have weight 0: 0-1 BFS behaves like regular BFS with all nodes at distance 0 from source.
- All edges have weight 1: 0-1 BFS behaves like regular BFS.
- Empty graph: single node, distance 0.
- Disconnected graph: unreachable nodes have INF distance.
- Source equals target: distance 0.
- Grid with all 0s: all cells reachable with cost 0.
- Grid with all 1s: cost = Manhattan distance.
- Graph with cycles: the distance check prevents infinite loops.

15. Variations

Variation	What Changes	When Used	Importance
0-k BFS (Dial's Algorithm)	Weights are 0, 1, ..., k	Use array of buckets	Moderate (CP)
1-2 BFS	Weights are 1 or 2	Can be reduced to 0-1 BFS by splitting edges	Low
BFS with State	0-1 BFS with additional state variable	More complex problems	Moderate
Minimum Cost to Make a Valid Path	Cost to change direction in a grid	0-1 BFS on direction graph	High

16. Related Algorithms / Data Structures

Structure	Relationship	How to Choose
BFS	For unweighted graphs	Use BFS for equal weights; 0-1 BFS for 0/1 weights
Dijkstra	For weighted graphs	Use Dijkstra for arbitrary positive weights; 0-1 BFS for 0/1
Deque	Core data structure	Use deque for push_front/push_back
Dial's Algorithm	Extension to 0-k BFS	Use when weights are small integers (0 to k)

17. Practice Problems

Easy

- **Minimum Time to Reach Destination** — Simple grid BFS. Easy.
- **Shortest Path in Binary Matrix** — LeetCode (1091) — BFS on grid. Medium.

Medium

- **Minimum Cost to Make at Least One Valid Path in a Grid** — LeetCode (1368) — 0-1 BFS. Hard.
- **Minimum Number of Flips to Convert Binary Matrix** — LeetCode (1284) — BFS with state. Hard.

Hard

- **Minimum Cost to Reach Destination** — 0-1 BFS on graph. Hard.
- **Shortest Path in a Grid with Obstacles Elimination** — LeetCode (1293) — BFS with state. Hard.

18. Interview Explanation

"0-1 BFS is a variation of BFS for graphs where edge weights are only 0 or 1. Instead of a regular queue, I use a deque. When I traverse a 0-weight edge, I push the neighbor to the front of the deque; for a 1-weight edge, I push to the back. This maintains the deque sorted by distance, so the first time I pop a node, I have the shortest distance to it. The time complexity is $O(V+E)$, which is faster than Dijkstra's $O((V+E) \log V)$ for this special case. It's commonly used in grid problems where some moves are free and others cost 1."

19. Revision Notes

- **Key idea:** Deque instead of queue/priority queue for 0/1 weights.
- **0-weight** → **push_front**, 1-weight → **push_back**.
- **Invariant:** Deque is sorted by distance.
- **Time:** $O(V+E)$, Space: $O(V)$.
- **Common trap:** Using regular queue for 0-1 BFS.
- **Grid variant:** Cost = value of target cell (0 or 1).

20. Final Cheat Sheet

0-1 BFS

When to use: Edge weights are 0 or 1 only

Data structure: Deque

Complexity: $O(V+E)$ time, $O(V)$ space

```
Key code:    if(dist[u] + w < dist[v]) {
              dist[v] = dist[u] + w;
              if(w == 0) dq.push_front(v);
              else dq.push_back(v);
            }
```

Edge cases: All 0-weights, all 1-weights, cycles

Trap: Don't use regular queue or priority queue

vs Dijkstra: $O(V+E)$ vs $O((V+E) \log V)$

Final Quick Reference

Complexity Comparison

Algorithm	Time	Space	Key Data Structure
Queue Basics	$O(1)$ per op	$O(n)$	Array + front/rear
Circular Queue	$O(1)$ per op	$O(\text{capacity})$	Circular array
Deque	$O(1)$ per op	$O(n)$	Circular array / linked list
BFS	$O(V+E)$	$O(V)$	Queue
Sliding Window Max	$O(n)$	$O(k)$	Monotonic deque
Monotonic Deque	$O(n)$	$O(k)$	Deque
First Negative in Window	$O(n)$	$O(k)$	Queue
Rotten Oranges	$O(N \times M)$	$O(N \times M)$	Queue (multi-source BFS)
Task Scheduler	$O(N)$	$O(1)$	Formula / Heap+Queue
0-1 BFS	$O(V+E)$	$O(V)$	Deque

Algorithm Selection Guide

Need FIFO? $\rightarrow$ Queue
Fixed-size buffer? $\rightarrow$ Circular Queue
Need both ends? $\rightarrow$ Deque
Shortest path (unweighted)? $\rightarrow$ BFS
Max in sliding window? $\rightarrow$ Monotonic Deque
First negative in window? $\rightarrow$ Queue of negative indices
Multi-source spread? $\rightarrow$ Multi-Source BFS
Tasks with cooldown? $\rightarrow$ Task Scheduler (Formula)
Shortest path (0/1 weights)? $\rightarrow$ 0-1 BFS (Deque)
Shortest path (any weights)? $\rightarrow$ Dijkstra